1
Pengantar Python

Python adalah bahasa pemrograman tingkat tinggi yang diciptakan oleh *Guido van Rossum,* seorang programmer asal Belanda. Python pertama kali dirilis pada tahun 1991, dan dikembangkan sebagai proyek sampingan selama libur Natal pada akhir tahun 1980-an. Nama "Python" sendiri diambil dari acara komedi Inggris, *Monty Python's Flying Circus*, yang merupakan acara favorit Guido van Rossum pada saat itu.

Berikut adalah sejarah singkat perkembangan Python dari awal hingga menjadi bahasa pemrograman populer saat ini:

1. **1980-an (Awal Pengembangan)**

   Guido van Rossum mulai mengembangkan Python pada akhir tahun 1980-an sebagai penerus dari bahasa ABC, sebuah bahasa yang dirancang untuk pemula dan mudah digunakan. Namun, Python dikembangkan dengan kemampuan untuk menangani berbagai kebutuhan pemrograman tingkat lanjut, termasuk sistem operasi dan pengembangan aplikasi jaringan.

2. **Python 1.0 (1991)**

   Python versi pertama dirilis pada tahun 1991. Versi ini memperkenalkan fitur-fitur dasar Python, seperti pengecekan tipe dinamis, kelas, fungsi, serta tipe data seperti list, dictionary, dan string. Python 1.0 sudah mendukung pemrograman berorientasi objek, fungsional, dan terstruktur, sehingga menjadi bahasa pemrograman yang serbaguna.

3. **Python 2.0 (2000)**

   Python 2.0 dirilis pada tahun 2000 dan merupakan lompatan besar dalam pengembangan bahasa ini. Fitur penting yang diperkenalkan di Python 2.0 adalah *garbage collection*, sistem untuk mengelola memori secara otomatis, dan *list comprehension*, yang memudahkan dalam memproses elemen-elemen di dalam list. Python 2.x tetap populer hingga bertahun-tahun dan menjadi fondasi pengembangan perangkat lunak di banyak perusahaan.

4. **Python 3.0 (2008)**

   Python 3.0, dirilis pada tahun 2008, membawa perubahan besar yang menyebabkan ketidakcocokan dengan Python 2.x. Tujuan utama Python 3.0 adalah untuk menghilangkan kekurangan yang ada pada Python 2.x, membuat bahasa ini lebih konsisten dan mudah dipahami. Beberapa perubahan penting di Python 3 adalah penyederhanaan fungsi *print*, penggunaan Unicode sebagai standar untuk string, dan penggantian cara penanganan angka integer.

5. **Era Python 3.x dan Popularitas Python**

   Setelah beberapa tahun, Python 3.x semakin diterima oleh komunitas, dan banyak pustaka serta framework yang diperbarui untuk mendukung Python 3. Python mulai dikenal sebagai bahasa yang sangat serbaguna dan banyak digunakan di bidang data science, kecerdasan buatan (AI), pengembangan web, dan otomasi sistem.

6. **Python sebagai Bahasa Populer (2010-an - Sekarang)**

   Sejak pertengahan 2010-an, Python menjadi salah satu bahasa pemrograman yang paling populer di dunia. Python dikenal dengan sintaksisnya yang sederhana dan mudah dipahami, sehingga sering dipilih oleh pemula. Selain itu, Python memiliki komunitas yang aktif dan dukungan pustaka yang luas, seperti NumPy dan Pandas untuk analisis data, TensorFlow dan PyTorch untuk pembelajaran mesin, serta Django dan Flask untuk pengembangan web.

Python terus berkembang, dan hingga saat ini Python tetap menjadi pilihan utama dalam berbagai aplikasi, dari pengembangan web hingga data science dan machine learning.

Kelebihan dan kerungan python dibandingkan bahasa C/C++

Python dan C/C++ adalah bahasa pemrograman yang sering dibandingkan karena memiliki kelebihan dan kekurangan yang berbeda sesuai dengan kebutuhan penggunanya. Berikut adalah beberapa kelebihan dan kekurangan Python dibandingkan dengan C/C++:

Kelebihan Python dibandingkan C/C++

1. **Sintaksis yang Mudah dan Sederhana**
   - Python memiliki sintaksis yang lebih sederhana dan mirip dengan bahasa manusia, sehingga lebih mudah dipahami, dipelajari, dan ditulis oleh pemula

maupun profesional. Di Python, tidak perlu mendeklarasikan tipe data secara eksplisit, dan penanganan memori dilakukan otomatis dengan *garbage collector*.

- C/C++ memiliki sintaksis yang lebih kompleks dan kaku. Struktur bahasa ini mengharuskan penggunaan tanda kurung kurawal {} dan titik koma ;, serta deklarasi tipe data untuk variabel.

2. **Produktivitas yang Tinggi**
   - Karena sintaksis Python yang sederhana, pengembangan dan debugging di Python menjadi lebih cepat. Hal ini membuat Python sangat cocok untuk *rapid prototyping*, pengembangan aplikasi sederhana, dan data science.
   - C/C++ membutuhkan kode yang lebih panjang dan waktu pengembangan lebih lama untuk aplikasi yang sama.
3. **Pustaka dan Ekosistem yang Kaya**
   - Python memiliki pustaka yang sangat luas, terutama untuk data science, kecerdasan buatan, *web development*, dan otomatisasi. Pustaka seperti NumPy, Pandas, TensorFlow, dan Django menjadikan Python sebagai bahasa utama di banyak bidang.
   - C++ juga memiliki pustaka yang banyak, tetapi pustaka yang ada di Python lebih unggul di bidang-bidang tertentu, seperti analisis data dan AI.
4. **Manajemen Memori Otomatis**
   - Python memiliki sistem *garbage collection* otomatis, yang secara otomatis membebaskan memori yang tidak digunakan. Ini mengurangi risiko kesalahan manajemen memori.
   - C++ mengharuskan pengembang untuk mengelola memori secara manual dengan mengalokasikan dan menghapus memori, yang dapat meningkatkan risiko kebocoran memori dan kesalahan lain yang berhubungan dengan memori.
5. **Kompatibilitas dan Keterbacaan Kode**
   - Kode Python yang sederhana membuatnya lebih mudah untuk di-*debug*, diuji, dan dipelihara. Python adalah bahasa yang interpretatif, yang memungkinkan menjalankan kode secara langsung tanpa perlu dikompilasi terlebih dahulu.
   - C++ perlu dikompilasi menjadi *binary* sebelum dieksekusi, dan keterbacaan kode sering kali tidak sebaik Python.

Kekurangan Python dibandingkan C/C++

1. **Kecepatan Eksekusi yang Lebih Lambat**
   - Python adalah bahasa interpretatif yang relatif lambat dibandingkan C/C++, yang merupakan bahasa terkompilasi. Python bukan pilihan terbaik untuk aplikasi yang membutuhkan kinerja tinggi, seperti game yang sangat intensif grafis atau sistem operasi.
   - C/C++ dikompilasi menjadi *machine code*, sehingga berjalan jauh lebih cepat dan sangat efisien untuk aplikasi yang membutuhkan kinerja optimal.
2. **Kontrol Lebih Rendah Terhadap Sistem**
   - Python tidak memberikan akses langsung ke perangkat keras dan kontrol rendah terhadap sistem, seperti pemrosesan memori dan perangkat input-output. Hal ini menjadikannya kurang ideal untuk pengembangan *low-level* dan aplikasi yang memerlukan optimasi tinggi.
   - C/C++ memungkinkan pengembang untuk mengakses dan mengontrol perangkat keras secara langsung, sehingga lebih cocok untuk aplikasi sistem, perangkat lunak tertanam (*embedded software*), dan perangkat keras.
3. **Kurangnya Dukungan untuk *Multithreading* Sebenarnya**
   - Python memiliki Global Interpreter Lock (GIL), yang membatasi eksekusi *thread* secara paralel di lingkungan multi-core. Hal ini membuat Python kurang optimal dalam memanfaatkan beberapa CPU core untuk tugas *multithreading*.
   - C++ mendukung *true multithreading* tanpa pembatasan seperti GIL, sehingga lebih efisien untuk aplikasi yang membutuhkan *multithreading* tingkat tinggi.
4. **Ukuran File Eksekusi yang Lebih Besar**
   - Program Python membutuhkan interpreter atau pustaka eksternal untuk dieksekusi, yang dapat meningkatkan ukuran file eksekusi akhir.
   - C++ menghasilkan file eksekusi yang lebih kecil karena langsung dikompilasi menjadi *machine code* tanpa perlu interpreter eksternal.
5. **Kurang Ideal untuk *Real-Time* dan *Embedded Systems***
   - Python jarang digunakan dalam sistem *embedded* atau aplikasi waktu nyata (*real-time applications*) karena keterbatasan kinerjanya.

- C++ adalah pilihan yang lebih baik untuk aplikasi ini, karena memungkinkan pengembangan program yang cepat dan responsif.

Kesimpulan

Python cocok untuk pengembangan aplikasi yang memerlukan produktivitas tinggi, prototyping cepat, dan analisis data. Di sisi lain, C/C++ lebih cocok untuk aplikasi yang membutuhkan kinerja tinggi dan kontrol rendah terhadap perangkat keras, seperti aplikasi sistem dan *embedded systems*.

TEKNIK KOMPUTER DAN JARINGAN
SMK NEGERI SUKASARI

2

Install dan Konfigurasi Python

Berikut adalah langkah-langkah untuk menginstall dan mengkonfigurasi Python di sistem operasi Windows, macOS, atau Linux:

Langkah 1: Download Python

1. **Buka Situs Resmi Python**

   Kunjungi https://www.python.org/downloads/ untuk mendownload versi terbaru Python.

2. **Pilih Versi Python**

   Pilih versi Python yang sesuai dengan sistem operasi Anda (Windows, macOS, atau Linux) dan klik "Download".

Langkah 2: Install Python

Di Windows

1. **Jalankan Installer**

   Buka file yang sudah didownload, dan pastikan untuk mencentang opsi *"Add Python to PATH"* di halaman awal installer.

2. **Pilih Install Now**

   Pilih opsi "Install Now" atau "Customize Installation" jika Anda ingin mengatur lokasi instalasi secara manual.

3. **Verifikasi Instalasi**

   Setelah instalasi selesai, buka Command Prompt dan ketik:

   ```
   python –version
   ```

   Anda akan melihat versi Python yang terinstal jika berhasil.

Di macOS

1. **Installer untuk macOS**

   Setelah mendownload file installer (file .pkg), buka file tersebut dan ikuti instruksi instalasi.

2. **Verifikasi Instalasi**

Setelah instalasi selesai, buka Terminal dan ketik:

```
python3 --version
```

Versi Python akan ditampilkan jika instalasi berhasil.

Di Linux

1. **Paket Manajer (APT untuk Ubuntu/Debian)**

   Di sebagian besar distribusi Linux, Python sudah terpasang secara default. Untuk memastikan versi terbaru, Anda dapat memperbarui Python dengan perintah berikut:

   ```
   sudo apt update
   sudo apt install python3
   ```

2. **Verifikasi Instalasi**

   Buka Terminal dan ketik:

   ```
   python3 --version
   ```

   Versi Python akan ditampilkan jika instalasi berhasil.

Langkah 3: Konfigurasi Lingkungan Python (Opsional)

1. **Menambahkan Python ke PATH (Windows Manual)**

   Jika Anda tidak mencentang *Add Python to PATH* saat instalasi, Anda perlu menambahkannya secara manual. Masuk ke:

   - *Control Panel > System and Security > System > Advanced system settings*
   - Klik pada "Environment Variables", cari variabel "Path", dan tambahkan lokasi direktori Python (misal, C:\Python39).

2. **Install PIP (Jika Belum Ada)**

   PIP adalah package manager untuk Python yang memungkinkan Anda menginstall pustaka pihak ketiga. PIP biasanya sudah terinstal bersama Python, tapi jika belum, Anda bisa menginstallnya dengan:

   ```
   python -m ensurepip --upgrade
   ```

3. **Membuat Virtual Environment (Opsional)**

   Virtual Environment (venv) memungkinkan Anda mengisolasi pustaka untuk setiap proyek. Untuk membuat *virtual environment*:

```
python -m venv nama_environment
```

Untuk mengaktifkan *environment*:

- **Windows**: nama_environment\Scripts\activate
- **macOS/Linux**: source nama_environment/bin/activate

4. **Menginstall Pustaka (Contoh)**

   Anda dapat menginstall pustaka atau modul tambahan dengan PIP. Misalnya, untuk menginstall pustaka requests:

```
pip install requests
```

Langkah 4: Pengujian Python

1. **Buka Terminal atau Command Prompt**

   Ketik python atau python3, dan Anda akan masuk ke mode interaktif Python (REPL) di mana Anda dapat mengetik perintah Python langsung.

2. **Coba Program Sederhana**

   Buat file Python baru (hello.py) dengan teks berikut:

```
print("Hello, World!")
```

   Kemudian jalankan dengan:

```
python hello.py
```

Itulah cara menginstal dan mengkonfigurasi Python di berbagai sistem operasi.

3

Dasar-Dasar Python

Dasar-dasar Python meliputi sintaksis, tipe data, operasi dasar, struktur kontrol, dan fungsi-fungsi dasar. Python adalah bahasa yang mudah dipelajari karena memiliki sintaksis yang sederhana. Berikut ini adalah beberapa konsep dasar dalam Python:

1. **Menulis Program Python**
   - **Print Statement**

     print() digunakan untuk menampilkan output ke layar.

     ```
     print("Hello, World!")
     ```
   - **Komentar**

     Komentar dimulai dengan tanda # dan digunakan untuk memberi keterangan pada kode.

     ```
     # Ini adalah komentar
     print("Komentar tidak dieksekusi")
     ```
2. **Variabel dan Tipe Data**

   Python adalah bahasa yang dinamis, artinya Anda tidak perlu mendeklarasikan tipe data variabel. Berikut adalah beberapa tipe data dasar:

   A. **Tipe Data Dasar**

   ```
   x = 5          # Integer
   y = 3.14       # Float
   name = "Alice"   # String
   is_active = True # Boolean
   ```

   B. **Tipe Data Koleksi**
   - **List**: Tipe data terurut yang dapat diubah.

     ```
     fruits = ["apple", "banana", "cherry"]
     ```
   - **Tuple**: Tipe data terurut yang tidak dapat diubah.

     ```
     coordinates = (10, 20)
     ```
   - **Set**: Koleksi tidak terurut yang tidak memiliki duplikasi.

```
unique_numbers = {1, 2, 3, 4}
```

C. **Dictionary**: Koleksi data pasangan kunci-nilai.

```
student = {"name": "Alice", "age": 20}
```

3. Operasi Dasar

A. Operasi Aritmatika

```
a = 10
b = 3
print(a + b)  # Penjumlahan
print(a - b)  # Pengurangan
print(a * b)  # Perkalian
print(a / b)  # Pembagian
print(a % b)  # Modulus
print(a ** b) # Pangkat
```

B. Operasi Perbandingan

Operator perbandingan digunakan untuk membandingkan dua nilai, dan hasilnya adalah nilai boolean (True atau False).

```
print(a == b) # Sama dengan
print(a != b) # Tidak sama dengan
print(a > b)  # Lebih besar
print(a < b)  # Lebih kecil
```

C. Operasi Logika

Operator logika digunakan untuk operasi boolean.

```
print(True and False) # False
print(True or False)  # True
print(not True)       # False
```

4. Struktur Kontrol

A. Percabangan (if-elif-else)

Digunakan untuk membuat keputusan berdasarkan kondisi.

```
x = 10
if x > 5:
    print("x lebih besar dari 5")
elif x == 5:
    print("x sama dengan 5")
else:
    print("x lebih kecil dari 5")
```

B. Perulangan (for dan while)

- **for** digunakan untuk iterasi dalam koleksi atau rentang angka.

```
for i in range(5):
    print(i)
```

- **while** digunakan untuk melakukan perulangan berdasarkan kondisi.

```
i = 0
while i < 5:
    print(i)
    i += 1
```

5. Fungsi

Fungsi adalah blok kode yang dapat digunakan kembali. Fungsi dideklarasikan dengan kata kunci def.

```
def greet(name):
    print(f"Hello, {name}!")

greet("Alice")  # Output: Hello, Alice!
```

Fungsi juga dapat mengembalikan nilai menggunakan return.

```
def add(a, b):
    return a + b

result = add(3, 4)
print(result)  # Output: 7
```

6. **Input dari Pengguna**

   Python memungkinkan kita untuk mengambil input dari pengguna menggunakan input().

```
name = input("Masukkan nama Anda: ")
print("Hello, " + name)
```

7. **Handling Error (Exception)**

   Python memiliki sistem untuk menangani kesalahan atau *exception*. Misalnya, try-except digunakan untuk menangkap dan menangani error.

```
try:
    angka = int(input("Masukkan angka: "))
    hasil = 10 / angka
    print("Hasil:", hasil)
except ZeroDivisionError:
    print("Tidak bisa membagi dengan nol!")
except ValueError:
    print("Input harus berupa angka!")
```

8. **Mengimport Modul**

   Python memiliki pustaka modul yang sangat luas. Modul dapat diimpor menggunakan import.

```
import math
print(math.sqrt(16))  # Output: 4.0
```

9. **List Comprehension**

   List comprehension adalah cara singkat untuk membuat list berdasarkan kondisi atau ekspresi tertentu.

```
squares = [x**2 for x in range(10)]
print(squares)  # Output: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

10. Membaca dan Menulis File

Python menyediakan API untuk membaca dan menulis file.

```
# Menulis ke file
with open("file.txt", "w") as file:
    file.write("Hello, World!")

# Membaca dari file
with open("file.txt", "r") as file:
    content = file.read()
    print(content)
```

Ini adalah dasar-dasar Python yang bisa membantu Anda memahami konsep utama dalam bahasa pemrograman ini. Selamat belajar dan mencoba!

TEKNIK JARINGAN
SMK NEGERI SUKASARI

4

Operators

Python memiliki berbagai jenis operator yang bisa digunakan untuk melakukan operasi aritmatika, perbandingan, logika, dan lainnya. Berikut adalah beberapa kategori operator dalam Python beserta contoh penggunaannya:

1. Operator Aritmatika

Digunakan untuk melakukan operasi matematika dasar.

Operator	Deskripsi	Contoh	Hasil
+	Penjumlahan	5 + 3	8
-	Pengurangan	5 - 3	2
*	Perkalian	5 * 3	15
/	Pembagian	5 / 3	1.666
%	Modulus	5 % 3	2
**	Pangkat	5 ** 3	125
//	Pembagian Bulat	5 // 3	1

```
a = 10
b = 3
print(a + b)  # 13
print(a - b)  # 7
print(a * b)  # 30
print(a / b)  # 3.3333...
print(a % b)  # 1
print(a ** b) # 1000
print(a // b) # 3
```

2. Operator Perbandingan

Digunakan untuk membandingkan dua nilai dan menghasilkan nilai boolean (True atau False).

Operator	Deskripsi	Contoh	Hasil
==	Sama dengan	5 == 3	False
!=	Tidak sama dengan	5 ! == 3	True
>	Lebih besar	5 > 3	True
<	Lebih kecil	5 < 3	False
>=	Lebih besar atau sama dengan	5 >= 3	True
<=	Lebih kecil atau sama dengan	5 <= 3	False

```
a = 10
b = 5
print(a == b)  # False
print(a != b)  # True
print(a > b)   # True
print(a < b)   # False
print(a >= b)  # True
print(a <= b)  # False
```

3. Operator Logika

Digunakan untuk operasi logika dan menghasilkan nilai boolean.

Operator	Deskripsi	Contoh	Hasil
and	Kedua kondisi benar	True and False	False
or	Salah satu kondisi benar	True or False	True
not	Membalik nilai	not True	False

```
x = True
y = False
print(x and y)  # False
print(x or y)   # True
print(not x)    # False
```

4. Operator Penugasan

Digunakan untuk memberikan nilai ke variabel.

Operator	Contoh	Ekivalen dengan
=	a = 5	a = 5
+=	a +=3	a= a + 3
-=	a -= 3	a = a - 3
*=	a *= 3	a = a * 3
/=	a /= 3	a = a * 3
%=	a %= 3	a = a % 3
//=	a //= 3	a = a // 3
**=	a ** = 3	a = a ** 3

```
a = 5
a += 3  # a = 8
a -= 2  # a = 6
a *= 4  # a = 24
a /= 2  # a = 12.0
print(a)
```

5. Operator Bitwise

Digunakan untuk operasi logika biner pada level bit.

Operator	Deskripsi	Contoh	Hasil
&	AND	5 & 3	1
|	OR	5 | 3	7
^	XOR	5 ^ 3	6
~	NOT	~5	-6
<<	Shift Left	5 << 1	10
>>	Shift Right	5 >> 1	2

```
a = 5       # 0101 in binary
b = 3       # 0011 in binary
print(a & b) # 0001 = 1
print(a | b) # 0111 = 7
print(a ^ b) # 0110 = 6
```

```
print(~a)    # 1010 = -6 (in two's complement representation)
print(a << 1) # 1010 = 10
print(a >> 1) # 0010 = 2
```

6. **Operator Keanggotaan**

Digunakan untuk memeriksa apakah suatu nilai ada dalam sebuah koleksi (seperti list, tuple, string, dll).

Operator	Deskripsi	Contoh	Hasil
in	Ada dalam koleksi	‘a’ in ‘apple’	True
not in	Tidak ada dalam koleksi	'b' not in 'apple'	True

```
fruits = ["apple", "banana", "cherry"]
print("apple" in fruits)   # True
print("grape" not in fruits) # True
```

7. **Operator Identitas**

Digunakan untuk memeriksa apakah dua objek memiliki referensi yang sama di memori.

Operator	Deskripsi	Contoh	Hasil
is	Objek yang sama	a is b	True atau False
is not	Objek yang berbeda	a is not b	True atau False

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]
print(a is b)   # True, karena a dan b menunjuk ke objek yang sama
print(a is c)   # False, karena a dan c adalah objek yang berbeda meskipun nilainya sama
print(a == c)   # True, karena nilai dari a dan c sama
```

Operator-operator ini adalah dasar dalam Python yang memungkinkan kita melakukan berbagai macam operasi dasar hingga kompleks dalam pemrograman.

TEKNIK KOMPUTER DAN JARINGAN
SMK NEGERI SUKASARI

5
Data Types

Python memiliki berbagai tipe data bawaan yang memudahkan pemrograman. Berikut ini adalah beberapa tipe data dasar di Python:

1. **Tipe Data Numerik**
   - **Integer (int)**: Tipe data untuk bilangan bulat positif atau negatif.

     ```
     x = 10
     y = -5
     ```

   - **Float (float)**: Tipe data untuk bilangan desimal atau bilangan real.

     ```
     pi = 3.14
     g = -9.81
     ```

   - **Complex (complex)**: Tipe data untuk bilangan kompleks, yang memiliki bagian real dan imajiner.

     ```
     z = 2 + 3j
     ```

2. **Tipe Data String (str)**

   Tipe data untuk menyimpan teks atau karakter. String dideklarasikan dengan tanda kutip tunggal (' ') atau kutip ganda (" ").

   ```
   name = "Alice"
   greeting = 'Hello, World!'
   ```

   String dapat diakses per karakter, digabungkan, dan memiliki banyak metode bawaan seperti upper(), lower(), dan replace().

   ```
   print(name.upper())  # ALICE
   ```

3. **Tipe Data Boolean (bool)**

   Tipe data untuk nilai logika True atau False.

   ```
   is_active = True
   is_open = False
   ```

Boolean sering digunakan dalam operasi perbandingan dan logika.

4. Tipe Data Koleksi

a. List

List adalah tipe data yang dapat menampung kumpulan nilai dan bersifat terurut serta dapat diubah. List dideklarasikan dengan tanda kurung siku [ ].

```
fruits = ["apple", "banana", "cherry"]
```

Anda dapat mengakses, menambahkan, menghapus, atau memodifikasi elemen dalam list.

```
fruits.append("orange")
fruits[1] = "grape"
```

b. Tuple

Tuple adalah tipe data terurut yang tidak dapat diubah. Dideklarasikan dengan tanda kurung ( ).

```
coordinates = (10, 20)
```

Karena bersifat tidak dapat diubah, tuple digunakan untuk menyimpan data yang tidak ingin diubah.

c. Set

Set adalah tipe data koleksi yang tidak terurut dan tidak memiliki elemen yang sama (unik). Dideklarasikan dengan tanda kurung kurawal { }.

```
unique_numbers = {1, 2, 3, 4}
```

Set digunakan untuk operasi himpunan seperti union, intersection, dan difference.

d. Dictionary

Dictionary adalah tipe data yang menyimpan pasangan kunci-nilai (key-value pairs) dan dideklarasikan dengan { }.

```
student = {"name": "Alice", "age": 20}
```

Nilai dalam dictionary dapat diakses menggunakan kunci.

```
print(student["name"])  # Alice
```

5. Tipe Data Khusus

a. None

None adalah tipe data khusus yang merepresentasikan ketidakberadaan nilai atau objek kosong.

```
x = None
```

b. Tipe Data Range

range adalah tipe data khusus yang merepresentasikan rentang angka, biasanya digunakan dalam perulangan.

```
numbers = range(5)  # Membuat range dari 0 hingga 4
```

6. Tipe Data Biner

a. Bytes

bytes digunakan untuk menyimpan data dalam bentuk biner.

```
b = b"hello"
```

b. Bytearray

bytearray mirip dengan bytes, tetapi bersifat mutable (dapat diubah).

```
ba = bytearray(5)  # Membuat array byte dengan panjang 5
```

7. Tipe Data Koleksi Lainnya

a. Frozen Set

Frozen Set adalah versi immutable dari set. Setelah didefinisikan, elemen dalam frozen set tidak dapat diubah.

```
fs = frozenset([1, 2, 3])
```

Contoh Penggunaan Tipe Data

```
# Numerik
x = 42              # int
pi = 3.14159        # float
z = 2 + 3j          # complex

# String
```

```
text = "Hello, Python!"

# Boolean
is_valid = True

# Koleksi
my_list = [1, 2, 3]                # list
my_tuple = (1, 2, 3)               # tuple
my_set = {1, 2, 3}                 # set
my_dict = {"name": "Alice", "age": 25}  # dictionary

# Khusus
nothing = None
```

Memahami tipe data ini penting karena memungkinkan kita untuk memilih tipe yang sesuai untuk data dan kebutuhan operasi tertentu dalam pemrograman Python.

SMK NEGERI SUKASARI

6
User Input

Di Python, **user input** adalah data yang dimasukkan oleh pengguna melalui keyboard. Python menyediakan fungsi bawaan input() untuk menerima input dari pengguna. Berikut adalah penjelasan dan contoh penggunaannya:

1. **Fungsi input()**

   Fungsi input() membaca input dari pengguna sebagai string secara default.

```
name = input("Masukkan nama Anda: ")
print(f"Hello, {name}!")
```

   Output:

```
Masukkan nama Anda: Alice
Hello, Alice!
```

2. **Membaca Input Angka**

   Jika ingin menerima angka (integer atau float), input string harus dikonversi menggunakan fungsi seperti int() atau float().

   a. **Input Integer**

```
age = int(input("Masukkan usia Anda: "))
print(f"Usia Anda adalah {age} tahun.")
```

   Output:

```
Masukkan usia Anda: 25
Usia Anda adalah 25 tahun.
```

   b. **Input Float**

```
height = float(input("Masukkan tinggi badan Anda (cm): "))
print(f"Tinggi Anda adalah {height} cm.")
```

   Output:

```
Masukkan tinggi badan Anda (cm): 175.5
Tinggi Anda adalah 175.5 cm.
```

3. **Input dengan Banyak Nilai (Menggunakan split())**

   Untuk menerima banyak nilai dalam satu baris, Anda bisa menggunakan metode split().

   a. **Input List dari Pengguna**

```
numbers = input("Masukkan beberapa angka, pisahkan dengan spasi: ").split()
print(f"Angka yang dimasukkan: {numbers}")
```

   Output:

```
Masukkan beberapa angka, pisahkan dengan spasi: 1 2 3 4
Angka yang dimasukkan: ['1', '2', '3', '4']
```

   b. **Konversi ke Tipe Data Lain**

```
numbers = list(map(int, input("Masukkan angka, pisahkan dengan spasi: ").split()))
print(f"Angka yang dimasukkan: {numbers}")
```

   Output:

```
Masukkan angka, pisahkan dengan spasi: 1 2 3 4
Angka yang dimasukkan: [1, 2, 3, 4]
```

4. **Mengatasi Error pada Input (Exception Handling)**

   Untuk menghindari kesalahan (misalnya pengguna memasukkan data yang salah), gunakan **try-except**.

```
try:
    num = int(input("Masukkan angka: "))
    print(f"Angka Anda: {num}")
except ValueError:
    print("Input harus berupa angka!")
```

   Output:

```
Masukkan angka: abc
Input harus berupa angka!
```

5. **Menggabungkan Input dengan Output**

   Gunakan format string (f-strings atau metode lainnya) untuk menggabungkan input dengan output.

   a. **Menggunakan +**

```
name = input("Masukkan nama Anda: ")
print("Hello, " + name + "!")
```

   b. **Menggunakan f-String**

```
name = input("Masukkan nama Anda: ")
print(f"Hello, {name}!")
```

   c. **Menggunakan format()**

```
name = input("Masukkan nama Anda: ")
print("Hello, {}!".format(name))
```

6. **Input dengan Batasan atau Validasi**

   Anda bisa memvalidasi input agar sesuai dengan kebutuhan.

   a. **Validasi Input Angka Positif**

```
while True:
    try:
        number = int(input("Masukkan angka positif: "))
        if number > 0:
            break
        else:
            print("Angka harus positif!")
    except ValueError:
        print("Input harus berupa angka!")
print(f"Angka yang dimasukkan: {number}")
```

7. **Contoh Lengkap Aplikasi Sederhana**

   a. **Kalkulator Sederhana**

```
print("Kalkulator Sederhana")
try:
```

```
    num1 = float(input("Masukkan angka pertama: "))
    num2 = float(input("Masukkan angka kedua: "))
    operator = input("Masukkan operator (+, -, *, /): ")

    if operator == "+":
        result = num1 + num2
    elif operator == "-":
        result = num1 - num2
    elif operator == "*":
        result = num1 * num2
    elif operator == "/":
        result = num1 / num2
    else:
        print("Operator tidak valid!")
        result = None

    if result is not None:
        print(f"Hasil: {result}")
except ValueError:
    print("Input harus berupa angka!")
```

Tips:

- Pastikan input divalidasi untuk menghindari error.
- Gunakan try-except untuk menangkap kesalahan.
- Konversikan input sesuai dengan tipe data yang diperlukan (int, float, dll).

7

Lists & Tuples

Lists & Tuples di Python adalah tipe data koleksi yang digunakan untuk menyimpan beberapa item dalam satu variabel. Meskipun keduanya mirip, terdapat perbedaan penting antara keduanya.

Lists

Ciri-ciri Lists:

- Bersifat **mutable** (dapat diubah setelah dibuat).
- Dideklarasikan menggunakan tanda kurung siku [ ].
- Dapat menyimpan berbagai jenis data (heterogen).

Contoh Penggunaan List

```
# Membuat List
fruits = ["apple", "banana", "cherry"]

# Mengakses Elemen
print(fruits[0])  # Output: apple

# Menambahkan Elemen
fruits.append("orange")
print(fruits)  # Output: ['apple', 'banana', 'cherry', 'orange']

# Mengubah Elemen
fruits[1] = "grape"
print(fruits)  # Output: ['apple', 'grape', 'cherry', 'orange']

# Menghapus Elemen
fruits.remove("cherry")
print(fruits)  # Output: ['apple', 'grape', 'orange']
```

```
# Looping melalui List
for fruit in fruits:
    print(fruit)

# List dengan tipe data campuran
mixed_list = [1, "hello", 3.14, True]
```

Metode Built-in pada List

Metode	Deskripsi	Contoh
append(x)	Menambahkan elemen ke akhir list	fruits.append("mango")
insert(i, x)	Menyisipkan elemen pada indeks tertentu	fruits.insert(1, "kiwi")
remove(x)	Menghapus elemen pertama yang memiliki nilai x	fruits.remove("banana")
pop([i])	Menghapus dan mengembalikan elemen pada indeks i	fruits.pop(1)
sort()	Mengurutkan elemen dalam list	fruits.sort()
reverse()	Membalik urutan elemen dalam list	fruits.reverse()
clear()	Menghapus semua elemen dalam list	fruits.clear()
count(x)	Menghitung jumlah elemen dengan nilai x	fruits.count("apple")
index(x)	Mengembalikan indeks dari elemen pertama dengan nilai x	fruits.index("apple")

Tuples

Ciri-ciri Tuples:

- Bersifat **immutable** (tidak dapat diubah setelah dibuat).
- Dideklarasikan menggunakan tanda kurung biasa ( ).
- Dapat menyimpan berbagai jenis data (heterogen).
- Digunakan untuk data yang tidak ingin diubah.

Contoh Penggunaan Tuple

```
# Membuat Tuple
coordinates = (10, 20, 30)
```

```
# Mengakses Elemen
print(coordinates[0])  # Output: 10

# Tuple dengan satu elemen (harus diakhiri dengan koma)
single_element = (5,)

# Tuple dengan tipe data campuran
mixed_tuple = (1, "hello", 3.14, True)

# Looping melalui Tuple
for coord in coordinates:
    print(coord)
```

Metode Built-in pada Tuple

Metode	Deskripsi	Contoh
count(x)	Menghitung jumlah elemen dengan nilai x	coordinates.count(20)
index(x)	Mengembalikan indeks dari elemen pertama dengan nilai x	coordinates.index(30)

Perbedaan Antara List dan Tuple

Aspek	List	Tuple
Mutable	Ya, dapat diubah	Tidak, immutable
Deklarasi	Menggunakan [ ]	Menggunakan ( )
Kecepatan	Lebih lambat	Lebih cepat
Penggunaan	Untuk data yang sering berubah	Untuk data yang konstan

Konversi Antara List dan Tuple

Anda dapat mengubah list menjadi tuple, dan sebaliknya.

```
# List ke Tuple
fruits = ["apple", "banana", "cherry"]
fruits_tuple = tuple(fruits)
print(fruits_tuple)  # Output: ('apple', 'banana', 'cherry')
```

```
# Tuple ke List
coordinates = (10, 20, 30)
coordinates_list = list(coordinates)
print(coordinates_list)  # Output: [10, 20, 30]
```

Contoh Aplikasi List dan Tuple

a. Menyimpan Daftar Item

```
shopping_list = ["milk", "bread", "eggs"]
```

b. Menyimpan Koordinat

```
location = (12.345, 67.890)  # Tidak akan berubah
```

List lebih fleksibel, sedangkan tuple digunakan untuk memastikan data tetap konsisten.

8

Dicts & Sets

Dictionaries dan Sets di Python adalah tipe data koleksi, tetapi masing-masing memiliki karakteristik yang unik dan berguna dalam berbagai situasi.

Dictionaries

Ciri-ciri Dictionaries:

- Berisi **pasangan key-value** (kunci-nilai).
- Kunci bersifat unik, sementara nilai bisa duplikat.
- Bersifat **mutable** (dapat diubah).
- Dideklarasikan menggunakan { }.

Contoh Penggunaan Dictionary

```
# Membuat Dictionary
student = {
    "name": "Alice",
    "age": 25,
    "major": "Computer Science"
}

# Mengakses Nilai
print(student["name"])  # Output: Alice

# Menambahkan Elemen
student["grade"] = "A"
print(student)  # Output: {'name': 'Alice', 'age': 25, 'major': 'Computer Science', 'grade': 'A'}

# Mengubah Nilai
student["age"] = 26
```

```
print(student)  # Output: {'name': 'Alice', 'age': 26, 'major': 'Computer Science', 'grade': 'A'}

# Menghapus Elemen
del student["major"]
print(student)  # Output: {'name': 'Alice', 'age': 26, 'grade': 'A'}
```

Metode Built-in pada Dictionary

Metode	Deskripsi	Contoh
`keys()`	Mengembalikan semua kunci dalam dictionary	`student.keys()`
`values()`	Mengembalikan semua nilai dalam dictionary	`student.values()`
`items()`	Mengembalikan pasangan key-value sebagai tuple	`student.items()`
`get(key, default)`	Mengakses nilai berdasarkan kunci, dengan nilai default jika tidak ada	`student.get("age", 0)`
`pop(key)`	Menghapus dan mengembalikan nilai berdasarkan kunci	`student.pop("grade")`
`update(dict2)`	Menambahkan atau mengupdate elemen dari dictionary lain	`student.update({"age": 30})`
`clear()`	Menghapus semua elemen dari dictionary	`student.clear()`

Looping pada Dictionary

```
for key, value in student.items():
    print(f"{key}: {value}")
```

Sets

Ciri-ciri Sets:

- Koleksi **elemen unik** (tidak ada duplikasi).

- Bersifat **tidak terurut** dan **tidak diindeks**.
- Bersifat **mutable** (dapat diubah), tetapi elemen di dalamnya harus **immutable**.
- Dideklarasikan menggunakan { } atau fungsi set().

Contoh Penggunaan Set

```
# Membuat Set
fruits = {"apple", "banana", "cherry"}

# Menambahkan Elemen
fruits.add("orange")
print(fruits)  # Output: {'orange', 'banana', 'apple', 'cherry'}

# Menghapus Elemen
fruits.remove("banana")
print(fruits)  # Output: {'orange', 'apple', 'cherry'}

# Mendeteksi Elemen
print("apple" in fruits)  # Output: True

# Membuat Set dari List
numbers = set([1, 2, 2, 3, 4])
print(numbers)  # Output: {1, 2, 3, 4}
```

Operasi pada Set

Operasi	Deskripsi	Contoh
`union()`	Menggabungkan dua set	`set1.union(set2)`
`intersection()`	Mengembalikan elemen yang sama di kedua set	`set1.intersection(set2)`
`difference()`	Elemen di set pertama tapi tidak di set kedua	`set1.difference(set2)`
`issubset()`	Mengembalikan `True` jika set adalah subset dari set lain	`set1.issubset(set2)`
`issuperset()`	Mengembalikan `True` jika set adalah superset dari set lain	`set1.issuperset(set2)`

Looping pada Set

```
for fruit in fruits:
```

```
print(fruit)
```

Frozen Set

Frozen set adalah versi **immutable** dari set.

```
frozen_fruits = frozenset(["apple", "banana", "cherry"])
print(frozen_fruits)
```

Perbedaan Antara Dictionary dan Set

Aspek	Dictionary	Set
Isi	Pasangan key-value	Hanya elemen unik
Mutable	Ya	Ya (kecuali frozen set)
Deklarasi	Menggunakan `{key: value}`	Menggunakan `{value}`
Akses	Elemen diakses melalui kunci	Tidak bisa diakses dengan indeks

Contoh Aplikasi Dictionary dan Set

a. Dictionary untuk Menyimpan Data Pasangan

```
student_scores = {"Alice": 90, "Bob": 85, "Charlie": 92}
```

b. Set untuk Menyimpan Data Unik

```
unique_ids = {101, 102, 103}
```

9

Loop

Loops dalam Python

Loops digunakan untuk menjalankan sekelompok instruksi secara berulang hingga suatu kondisi tertentu terpenuhi. Python mendukung dua jenis loop utama: **for loop** dan **while loop**.

1. for Loop

for loop digunakan untuk iterasi melalui elemen dalam koleksi (seperti list, tuple, dictionary, set, atau string).

Sintaks

```
for item in iterable:
    # blok kode
```

Contoh Penggunaan

a. Iterasi Melalui List

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

Output:

```
apple
banana
cherry
```

b. Iterasi dengan Range

```
for i in range(5):
    print(i)
```

Output:

```
0
1
```

```
2
3
4
```

c. Iterasi dengan Index

```
fruits = ["apple", "banana", "cherry"]
for index, fruit in enumerate(fruits):
    print(f"{index}: {fruit}")
```

Output:

```
0: apple
1: banana
2: cherry
```

d. Iterasi Melalui String

```
word = "Python"
for char in word:
    print(char)
```

Output:

```
P
y
t
h
o
n
```

2. while Loop

while loop digunakan untuk menjalankan blok kode selama suatu kondisi bernilai True.

Sintaks

```
while condition:
    # blok kode
```

Contoh Penggunaan

a. Loop dengan Kondisi

```
count = 0
while count < 5:
    print(count)
    count += 1
```

Output:

```
0
1
2
3
4
```

b. Loop dengan Input

```
password = ""
while password != "12345":
    password = input("Masukkan password: ")
print("Akses diberikan!")
```

3. Control Statements dalam Loop

a. break

Menghentikan loop sepenuhnya.

```
for i in range(10):
    if i == 5:
        break
    print(i)
```

Output:

```
0
1
2
3
4
```

b. continue

Melewati iterasi saat ini dan melanjutkan ke iterasi berikutnya.

```
for i in range(5):
    if i == 2:
        continue
    print(i)
```

Output:

```
0
1
3
4
```

c. else dengan Loop

Blok else pada loop akan dijalankan jika loop selesai tanpa terkena break.

```
for i in range(5):
    print(i)
else:
    print("Loop selesai tanpa break!")
```

Output:

```
0
1
2
3
4
Loop selesai tanpa break!
```

4. Nested Loops

Loop di dalam loop disebut **nested loop**.

Contoh

```
for i in range(3):
    for j in range(2):
        print(f"i={i}, j={j}")
```

Output:

```
i=0, j=0
i=0, j=1
i=1, j=0
i=1, j=1
i=2, j=0
i=2, j=1
```

5. **Infinite Loop**

Hati-hati dengan **infinite loop** karena dapat membuat program tidak berhenti.

Contoh Infinite Loop

```
while True:
    print("Ini loop tak terbatas!")
```

Untuk menghentikannya, Anda bisa menekan **Ctrl + C**.

Contoh Aplikasi Loops

a. Mencari Bilangan Prima

```
n = 10
for num in range(2, n + 1):
    for i in range(2, num):
        if num % i == 0:
            break
    else:
        print(num)
```

b. Menampilkan Pola

```
rows = 5
for i in range(1, rows + 1):
    print("*" * i)
```

Output:

```
*
**
***
```

```
****
*****
```

Loops sangat berguna untuk mengotomasi tugas-tugas yang berulang dalam berbagai skenario!

TEKNIK KOMPUTER DAN JARINGAN
SMK NEGERI SUKASARI

10
Perintah IF

Perintah if dalam Python digunakan untuk **percabangan** atau **pengambilan keputusan** dalam program. Dengan menggunakan if, kamu bisa membuat program yang **melakukan sesuatu hanya jika kondisi tertentu terpenuhi**.

Pengertian Perintah if

if adalah struktur kontrol yang digunakan untuk mengevaluasi suatu kondisi (bernilai **True** atau **False**). Jika kondisi tersebut benar (**True**), maka blok kode di dalam if akan dijalankan.

Fungsi dan Kegunaan if

1. Mengontrol alur program berdasarkan kondisi.
2. Menjalankan perintah hanya ketika kondisi tertentu terpenuhi.
3. Digunakan untuk membuat logika seperti:
   - Pengambilan keputusan
   - Validasi data
   - Percabangan program

Contoh :

```
umur = 20

if umur >= 18:
   print("Kamu sudah dewasa")
```

Penjelasan:

- Jika nilai umur lebih besar atau sama dengan 18, maka akan muncul teks “Kamu sudah dewasa”.
- Kalau tidak, maka tidak terjadi apa-apa (karena tidak ada else).

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

Struktur Dasar Perintah if

```
if kondisi:
    # blok kode yang dijalankan jika kondisi True
```

Kamu juga bisa menambahkan:

- else: jika kondisi **tidak terpenuhi**
- elif: jika ada **kondisi lain** yang ingin diperiksa

Contoh lengkap:

```
nilai = 75

if nilai >= 90:
    print("Nilai A")
elif nilai >= 75:
    print("Nilai B")
else:
    print("Nilai C")
```

Latihan 1: Cek Bilangan Positif atau Negatif

Soal: Buat program yang mengecek apakah sebuah bilangan adalah positif, negatif, atau nol.

```
angka = int(input("Masukkan angka: "))

if angka > 0:
    print("Angka positif")
elif angka < 0:
    print("Angka negatif")
else:
    print("Angka nol")
```

Latihan 2: Cek Kelulusan

Soal: Buat program yang menerima nilai siswa dan mencetak "Lulus" jika nilainya 60 ke atas, dan "Tidak Lulus" jika di bawah 60.

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

```
nilai = int(input("Masukkan nilai: "))

if nilai >= 60:
    print("Lulus")
else:
    print("Tidak Lulus")
```

Latihan 3: Menentukan Kategori Usia

Soal: Buat program yang menerima input usia dan mengkategorikannya:

- Anak-anak (0–12)
- Remaja (13–17)
- Dewasa (18–59)
- Lansia (60 ke atas)

```
usia = int(input("Masukkan usia: "))

if usia <= 12:
    print("Anak-anak")
elif usia <= 17:
    print("Remaja")
elif usia <= 59:
    print("Dewasa")
else:
    print("Lansia")
```

Latihan 4: Cek Angka Ganjil atau Genap

Soal: Buat program yang menerima angka dan menentukan apakah itu ganjil atau genap.

```
angka = int(input("Masukkan angka: "))

if angka % 2 == 0:
    print("Angka genap")
else:
    print("Angka ganjil")
```

Soal Essay 1: Cek Suhu Air

Buatlah program Python yang meminta input suhu air (dalam derajat Celcius) dari pengguna, kemudian tampilkan:

- "Air dalam keadaan beku" jika suhu di bawah 0°C
- "Air dalam keadaan normal" jika suhu antara 0°C sampai 100°C
- "Air dalam keadaan mendidih" jika suhu di atas 100°C

Soal Essay 2: Menentukan Tahun Kabisat

Buatlah program Python yang menerima input tahun dari pengguna, lalu tentukan apakah tahun tersebut merupakan tahun kabisat atau bukan.

Petunjuk:

- Tahun kabisat adalah tahun yang habis dibagi 4,
- Kecuali tahun yang habis dibagi 100 **bukan** kabisat,
- **Kecuali lagi** jika juga habis dibagi 400, maka tetap kabisat.

Soal Essay 3: Penilaian Ujian

Buatlah program yang meminta input nilai ujian dari pengguna, lalu tampilkan kategori nilai:

- Nilai ≥ 90 → "Sangat Baik"
- Nilai ≥ 75 → "Baik"
- Nilai ≥ 60 → "Cukup"
- Nilai < 60 → "Kurang"

Soal Essay 4: Kalkulator Sederhana

Buatlah program yang meminta:

- Dua buah angka dari pengguna
- Satu operator (misalnya: +, -, *, /)

Kemudian program menampilkan hasil operasi berdasarkan operator yang dipilih.

Contoh:

```
Masukkan angka pertama: 10
Masukkan angka kedua: 5
```

```
Masukkan operator (+, -, *, /): *
Hasil: 50
```

TEKNIK KOMPUTER DAN JARINGAN
SMK NEGERI SUKASARI

11

Fungi Aritmatic dan Math

Dalam Python, terdapat dua jenis fungsi yang sering digunakan untuk operasi numerik:

1. **Operator Aritmatika (Arithmetic Operators)**
2. **Fungsi Matematika (Math Functions)**

1. Operator Aritmatika

Operator ini digunakan untuk operasi dasar matematika:

Operator	Nama	Contoh (a = 10, b = 3)	Hasil
+	Penjumlahan	a + b	13
-	Pengurangan	a - b	7
*	Perkalian	a * b	30
/	Pembagian	a / b	3.333
//	Pembagian Bulat	a // b	3
%	Modulus (Sisa)	a % b	1
**	Perpangkatan	a ** b	1000

2. Fungsi Matematika (Modul math)

Python menyediakan modul math yang berisi banyak fungsi matematika.

Pertama, import dulu:

```
import math
```

Fungsi-Fungsi Umum:

Fungsi	Keterangan	Contoh
math.sqrt(x)	Akar kuadrat	math.sqrt(16) → 4.0
math.pow(x, y)	Pangkat (x^y)	math.pow(2, 3) → 8.0
math.floor(x)	Pembulatan ke bawah	math.floor(3.7) → 3
math.ceil(x)	Pembulatan ke atas	math.ceil(3.2) → 4
math.fabs(x)	Nilai absolut	math.fabs(-5) → 5.0
math.factorial(x)	Faktorial	math.factorial(5) → 120

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

math.log(x)	Logaritma natural (ln)	math.log(10)
math.log10(x)	Logaritma basis 10	math.log10(100) → 2.0
math.sin(x)	Sinus (x dalam radian)	math.sin(math.pi/2) → 1.0
math.radians(x)	Konversi derajat ke radian	math.radians(180) → π

Contoh program Python yang menggunakan **operator aritmatika** dan **fungsi dari modul math**:

Contoh Program Lengkap:

```
import math

# Angka awal
a = 10
b = 3

# Operator Aritmatika
print("== Operator Aritmatika ==")
print(f"{a} + {b} = {a + b}")
print(f"{a} - {b} = {a - b}")
print(f"{a} * {b} = {a * b}")
print(f"{a} / {b} = {a / b}")
print(f"{a} // {b} = {a // b}")
print(f"{a} % {b} = {a % b}")
print(f"{a} ** {b} = {a ** b}")

print("\n== Fungsi Math ==")
# Fungsi Matematika
x = 16
y = -7
z = 100

print(f"math.sqrt({x}) = {math.sqrt(x)}")       # Akar kuadrat
```

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

```
print(f"math.pow(2, 3) = {math.pow(2, 3)}")      # Pangkat
print(f"math.floor(7.8) = {math.floor(7.8)}")    # Pembulatan ke bawah
print(f"math.ceil(7.1) = {math.ceil(7.1)}")      # Pembulatan ke atas
print(f"math.fabs({y}) = {math.fabs(y)}")        # Nilai absolut
print(f"math.factorial(5) = {math.factorial(5)}")# Faktorial
print(f"math.log(10) = {math.log(10):.4f}")      # Log natural
print(f"math.log10({z}) = {math.log10(z)}")      # Log basis 10
print(f"math.sin(math.pi/2) = {math.sin(math.pi/2)}")  # Sinus dari 90°
print(f"math.radians(180) = {math.radians(180)}")      # Derajat ke radian
```

Output Program:

```
== Operator Aritmatika ==
10 + 3 = 13
10 - 3 = 7
10 * 3 = 30
10 / 3 = 3.3333333333333335
10 // 3 = 3
10 % 3 = 1
10 ** 3 = 1000

== Fungsi Math ==
math.sqrt(16) = 4.0
math.pow(2, 3) = 8.0
math.floor(7.8) = 7
math.ceil(7.1) = 8
math.fabs(-7) = 7.0
math.factorial(5) = 120
math.log(10) = 2.3026
math.log10(100) = 2.0
math.sin(math.pi/2) = 1.0
math.radians(180) = 3.141592653589793
```

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

Berikut ini beberapa **soal latihan** tentang operator aritmatika dan fungsi math di Python. Cocok buat latihan pemahaman dasar

Latihan 1 – Operator Aritmatika Dasar

Soal: Buat program Python yang menerima dua input angka dari user, lalu tampilkan hasil:

1. Penjumlahan
2. Pengurangan
3. Perkalian
4. Pembagian
5. Modulus
6. Perpangkatan

Contoh Input:

```
Masukkan angka pertama: 8
Masukkan angka kedua: 2
```

Contoh Output:

```
Hasil penjumlahan: 10
Hasil pengurangan: 6
Hasil perkalian: 16
Hasil pembagian: 4.0
Hasil modulus: 0
Hasil perpangkatan: 64
```

Latihan 2 – Fungsi Math Sederhana

Soal: Buat program Python yang menerima input angka, lalu tampilkan:

1. Akar kuadrat
2. Logaritma natural
3. Logaritma basis 10
4. Nilai sinus dari 90 derajat
5. Nilai absolut

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

Contoh Output:

```
Masukkan angka: 25
Akar kuadrat: 5.0
Log natural: 3.2189
Log basis 10: 1.3979
Sinus(90°): 1.0
Nilai absolut: 25
```

Latihan 3 – Kombinasi Math + Aritmatika

Soal: Buat program yang:

1. Menghitung luas lingkaran berdasarkan jari-jari input dari user.
2. Menghitung keliling lingkaran.
3. Membulatkan hasil luas ke atas dan ke bawah.

Rumus:

- Luas = $\pi \times r^2$
- Keliling = $2 \times \pi \times r$

Contoh Output:

```
Masukkan jari-jari: 4
Luas lingkaran: 50.265
Keliling: 25.1327
Luas dibulatkan ke bawah: 50
Luas dibulatkan ke atas: 51
```

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

12

Operator Logika

Dalam Python, **operator logika** digunakan untuk menggabungkan pernyataan (ekspresi) kondisional. Operator logika ini sering digunakan dalam pernyataan seperti if, while, dan sebagainya.

Berikut adalah **operator logika** utama dalam Python:

Operator	Nama	Contoh Penggunaan	Hasil
and	Logika DAN	True and False	False (keduanya harus True)
or	Logika ATAU	True or False	True (salah satu True)
not	Logika NOT	not True	False (kebalikan nilai)

Penjelasan Singkat:

1. **and**: Mengembalikan True jika **kedua** ekspresi adalah True.

```
a = 10
b = 5
if a > 0 and b > 0:
    print("Keduanya positif")
```

2. **or**: Mengembalikan True jika **salah satu** ekspresi adalah True.

```
a = -1
b = 5
if a > 0 or b > 0:
    print("Salah satu positif")
```

3. **not**: Membalikkan nilai boolean.

```
is_online = False
if not is_online:
    print("User sedang offline")
```

latihan soal sederhana tentang operator logika di Python

Soal 1:

```
x = 7
```

```
y = 3

if x > 5 and y < 5:
    print("Benar")
else:
    print("Salah")
```

Pertanyaan: Apa output dari kode di atas?

Soal 2:

```
a = 10
b = -2

if a < 0 or b < 0:
    print("Ada yang negatif")
else:
    print("Keduanya positif")
```

Soal 3:

```
is_admin = True

if not is_admin:
    print("Bukan admin")
else:
    print("Admin")
```

Soal 4:

```
nilai = 85

if nilai >= 80 and nilai <= 100:
    print("Nilai A")
```

Pertanyaan: Apakah akan mencetak sesuatu?

Soal Essay 1:

Jelaskan perbedaan antara operator and dan or dalam Python. Berikan masing-masing contoh kode dan jelaskan hasilnya.

Soal Essay 2:

Diberikan variabel berikut:

```
umur = 20
punya_sim = True
```

Buatlah program dengan logika:

- Jika umur lebih dari atau sama dengan 17 **dan** punya SIM, tampilkan "Boleh mengemudi"
- Jika tidak, tampilkan "Tidak boleh mengemudi"

Soal Essay 3:

Apa hasil output dari kode berikut? Jelaskan kenapa!

```
x = False
y = True
z = not x and y
print(z)
```

Soal Essay 4:

Buatlah sebuah program Python yang memeriksa apakah sebuah angka berada di antara 10 sampai 50 (inklusif). Jika iya, cetak "Angka dalam rentang", jika tidak, cetak "Angka di luar rentang".

Gunakan **operator logika** dalam programmu.

13
String Methods

String methods pada Python adalah **fungsi-fungsi bawaan (built-in)** yang digunakan untuk memanipulasi dan mengelola data bertipe string. String methods biasanya digunakan untuk mengubah, mencari, memisah, mengganti, atau memformat string tanpa harus membuat logika manual dari awal.

String di Python adalah objek dari kelas str, sehingga memiliki berbagai metode (methods) yang bisa dipanggil menggunakan tanda titik (dot).

Fungsi-Fungsi String Methods (Beberapa Contoh Populer):

Method	**Fungsi**
upper()	Mengubah semua huruf dalam string menjadi **huruf besar**.
lower()	Mengubah semua huruf dalam string menjadi **huruf kecil**.
capitalize()	Mengubah huruf pertama menjadi huruf besar, sisanya kecil.
title()	Mengubah setiap huruf pertama dari kata menjadi kapital.
strip()	Menghapus spasi (atau karakter tertentu) di awal dan akhir string.
replace(old, new)	Mengganti teks tertentu dengan teks baru.
split(separator)	Memecah string menjadi list berdasarkan pemisah tertentu.
join(list)	Menggabungkan list menjadi satu string, dengan pemisah tertentu.
find(substring)	Mengembalikan posisi pertama substring, atau -1 jika tidak ditemukan.
startswith(text)	Mengecek apakah string diawali dengan teks tertentu.
endswith(text)	Mengecek apakah string diakhiri dengan teks tertentu.
isalpha()	Mengecek apakah string hanya terdiri dari huruf.
isdigit()	Mengecek apakah string hanya terdiri dari angka.

Contoh Penggunaan:

```
teks = "  Belajar Python itu Mudah  "
```

```
print(teks.upper())       # BELAJAR PYTHON ITU MUDAH
print(teks.lower())       # belajar python itu mudah
print(teks.strip())       # "Belajar Python itu Mudah"
print(teks.replace("Mudah", "Seru"))  # Belajar Python itu Seru
print(teks.split())       # ['Belajar', 'Python', 'itu', 'Mudah']
print("python".capitalize())  # Python
```

Berikut adalah **daftar lengkap** beberapa **string methods** di Python beserta **penjelasan** dan **contoh penggunaannya**:

1. upper()

Mengubah semua karakter dalam string menjadi huruf besar.

```
teks = "python"
print(teks.upper())  # Output: "PYTHON"
```

2. lower()

Mengubah semua karakter dalam string menjadi huruf kecil.

```
teks = "PYTHON"
print(teks.lower())  # Output: "python"
```

3. capitalize()

Mengubah huruf pertama menjadi huruf besar dan sisanya menjadi huruf kecil.

python

```
teks = "python"
print(teks.capitalize())  # Output: "Python"
```

4. title()

Mengubah setiap huruf pertama dari setiap kata menjadi kapit

```
teks = "belajar python mudah"
print(teks.title())  # Output: "Belajar Python Mudah"
```

5. strip()

Menghapus spasi atau karakter tertentu di awal dan akhir string.

```
teks = "  python  "
print(teks.strip())  # Output: "python"
```

6. replace(old, new)

Mengganti semua kejadian old dengan new.

```
teks = "belajar python"
print(teks.replace("python", "java"))  # Output: "belajar java"
```

7. split(separator)

Memecah string menjadi list berdasarkan separator yang diberikan.

```
teks = "belajar python mudah"
print(teks.split())  # Output: ['belajar', 'python', 'mudah']
```

8. join(list)

Menggabungkan elemen-elemen dari list menjadi string dengan separator tertentu.

```
list_teks = ['belajar', 'python', 'mudah']
print(" ".join(list_teks))  # Output: "belajar python mudah"
```

9. find(substring)

Mengembalikan indeks pertama dari substring jika ditemukan, atau -1 jika tidak ditemukan.

```
teks = "belajar python"
print(teks.find("python"))  # Output: 8
print(teks.find("java"))    # Output: -1
```

10. startswith(prefix)

Memeriksa apakah string diawali dengan prefix tertentu.

```
teks = "belajar python"
print(teks.startswith("belajar"))  # Output: True
```

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

```
print(teks.startswith("java"))     # Output: False
```

11. endswith(suffix)

Memeriksa apakah string diakhiri dengan suffix tertentu.

```
teks = "belajar python"
print(teks.endswith("python"))  # Output: True
print(teks.endswith("java"))    # Output: False
```

12. isalpha()

Memeriksa apakah string hanya terdiri dari huruf.

```
teks = "python"
print(teks.isalpha())  # Output: True
teks2 = "python3"
print(teks2.isalpha())  # Output: False
```

13. isdigit()

Memeriksa apakah string hanya terdiri dari angka.

```
teks = "12345"
print(teks.isdigit())  # Output: True
teks2 = "123a45"
print(teks2.isdigit())  # Output: False
```

14. isspace()

Memeriksa apakah string hanya terdiri dari spasi.

```
teks = "    "
print(teks.isspace())  # Output: True
teks2 = "   python  "
print(teks2.isspace())  # Output: False
```

15. isnumeric()

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

Memeriksa apakah string hanya terdiri dari karakter numerik (termasuk angka dan simbol lain seperti angka Romawi).

```
teks = "12345"
print(teks.isnumeric())  # Output: True
```

16. isupper()

Memeriksa apakah seluruh karakter dalam string adalah huruf besar.

```
teks = "PYTHON"
print(teks.isupper())  # Output: True
```

17. islower()

Memeriksa apakah seluruh karakter dalam string adalah huruf kecil.

```
teks = "python"
print(teks.islower())  # Output: True
```

18. swapcase()

Mengubah huruf besar menjadi huruf kecil dan sebaliknya.

```
teks = "Python"
print(teks.swapcase())  # Output: "pYTHON"
```

19. zfill(width)

Menambahkan nol di depan string hingga panjangnya mencapai width.

```
teks = "42"
print(teks.zfill(5))  # Output: "00042"
```

20. ljust(width, fillchar)

Menambah karakter tertentu di sebelah kanan string hingga panjangnya mencapai width.

```
teks = "python"
print(teks.ljust(10, "-"))  # Output: "python----"
```

TEKNIK KOMPUTER DAN JARINGAN
SMK NEGERI SUKASARI

21. rjust(width, fillchar)

Menambah karakter tertentu di sebelah kiri string hingga panjangnya mencapai width.

```
teks = "python"
print(teks.rjust(10, "-"))  # Output: "----python"
```

22. center(width, fillchar)

Menambahkan karakter tertentu di kiri dan kanan string agar panjangnya mencapai width dan string dipusatkan.

```
teks = "python"
print(teks.center(10, "-"))  # Output: "--python--"
```

Kesimpulan

String methods di Python sangat berguna untuk berbagai manipulasi string, mulai dari konversi huruf, pemisahan string, hingga pemeriksaan kondisi tertentu pada string. Kamu bisa menggabungkan beberapa metode ini untuk mendapatkan hasil yang lebih kompleks dan sesuai kebutuhan.

TEKNIK KOMPUTER DAN JARINGAN
SMK NEGERI SUKASARI

14

String Indexing

String indexing dalam Python adalah proses mengakses karakter individual dalam sebuah string menggunakan **indeks (index)**. String pada Python bersifat **immutable**, artinya kamu tidak bisa mengubah karakter di dalam string setelah string itu dibuat, tapi kamu bisa **mengakses** tiap karakter menggunakan indeks.

Cara Kerja Indexing

- **Indeks dimulai dari 0** untuk karakter pertama, 1 untuk karakter kedua, dan seterusnya.
- Python juga mendukung **indeks negatif**, di mana -1 merujuk ke karakter terakhir, -2 ke karakter kedua dari belakang, dan seterusnya.

Contoh:

```
teks = "Python"
print(teks[0])   # Output: P
print(teks[3])   # Output: h
print(teks[-1])  # Output: n
```

Fungsi String Indexing

String indexing berfungsi untuk:

1. **Mengakses karakter tertentu dalam string**

```
kata = "Belajar"
print(kata[2])  # Output: l
```

2. Digunakan dalam kombinasi dengan slicing untuk mengambil substring

```
kalimat = "Belajar Python"
print(kalimat[0:7])  # Output: Belajar
```

3. Membalik string menggunakan indeks negatif

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

```
print(kalimat[::-1])  # Output: nohtyP rajaleB
```

4. Digunakan dalam perulangan untuk memproses tiap karakter

```
for i in range(len(kalimat)):
    print(kalimat[i])
```

String Slicing

Slicing adalah teknik untuk **mengambil bagian (substring)** dari string berdasarkan indeks tertentu. Slicing sangat berguna ketika kamu hanya ingin mengambil sebagian dari isi string.

Sintaks Slicing

```
string[start:stop:step]
```

- start: indeks awal (opsional, default-nya 0)
- stop: indeks akhir **(tidak termasuk karakter di indeks ini)**
- step: langkah (opsional, default-nya 1)

Contoh-Contoh Slicing

Misalnya kita punya string:

```
teks = "Belajar Python"
```

1. Mengambil karakter dari indeks 0 sampai 6

```
print(teks[0:7])   # Output: Belajar
```

2. Mengambil karakter dari indeks 8 sampai akhir

```
print(teks[8:])    # Output: Python
```

3. Mengambil karakter dari awal sampai sebelum indeks 7

```
print(teks[:7])    # Output: Belajar
```

4. Mengambil seluruh string dengan langkah 2

```
print(teks[::2])   # Output: BljrPto
```

5. Membalik string

```
print(teks[::-1])  # Output: nohtyP rajaleB
```

Tips Slicing

- Kamu bisa pakai **slicing dengan step negatif** untuk balik string atau melompat mundur.
- Kalau kamu lupa stop, Python akan otomatis ambil sampai akhir string.
- Kalau lupa start, Python mulai dari awal.

Latihan Soal 1

Diberikan string:

```
teks = "Programmer Python"
```

Ambil kata "Programmer" saja menggunakan slicing.

Latihan Soal 2

Dari string yang sama:

```
teks = "Programmer Python"
```

Ambil kata "Python" saja menggunakan slicing.

Latihan Soal 3

Ambil 4 karakter pertama dari string berikut:

```
teks = "BelajarKoding"
```

Latihan Soal 4

Ambil setiap **karakter kedua** dari string berikut:

```
teks = "BelajarKoding"
```

Hint: gunakan step

Latihan Soal 5

Balikkan string berikut:

```
teks = "AkuSukaPython"
```

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

15
Function

Pengertian Function

Function (fungsi) dalam Python adalah **sekelompok pernyataan (kode)** yang dijalankan bersama-sama untuk melakukan **tugas tertentu**. Function digunakan untuk **mengorganisir kode agar lebih terstruktur, mudah dibaca, dan dapat digunakan kembali** (reusable).

Function biasanya dibuat dengan kata kunci def, diikuti dengan nama function dan tanda kurung.

Contoh dasar:

```
def sapa():
    print("Halo, selamat datang di Python!")
```

Untuk memanggilnya:

```
sapa()
```

Fungsi dari Function dalam Python

1. **Menghindari pengulangan kode**
   Jika kamu punya blok kode yang sering digunakan, kamu bisa memasukkannya ke dalam function dan memanggilnya kapan saja.
2. **Mempermudah debugging**
   Kode yang modular lebih mudah dilacak jika terjadi kesalahan.
3. **Meningkatkan keterbacaan kode**
   Membuat kode lebih ringkas, rapi, dan mudah dipahami.
4. **Memungkinkan pembagian kerja dalam tim**
   Setiap orang bisa mengerjakan function tertentu tanpa mengganggu bagian lain.
5. **Mendukung prinsip DRY (Don't Repeat Yourself)**
6. Menyusun kode agar tidak menulis hal yang sama berulang kali.

Function dengan Parameter dan Nilai Kembali (Return Value)

Contoh:

```
def tambah(a, b):
    return a + b

hasil = tambah(3, 4)
print(hasil)  # Output: 7
```

- a dan b adalah parameter
- return digunakan untuk mengembalikan nilai

Parameter dan Argumen

- **Parameter** adalah variabel yang ditulis di dalam tanda kurung saat mendefinisikan function.
- **Argumen** adalah nilai yang diberikan ke parameter saat function dipanggil.

Contoh:

```
def salam(nama):
    print(f"Halo, {nama}!")

salam("Andi")  # Output: Halo, Andi!
```

Return Statement

Kalau kamu ingin function mengembalikan nilai, kamu pakai return.

Contoh:

```
def luas_persegi_panjang(panjang, lebar):
    return panjang * lebar

hasil = luas_persegi_panjang(5, 3)
print("Luas:", hasil)
```

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

Function dengan Default Parameter

Kamu bisa memberi nilai default ke parameter, jadi kalau tidak ada argumen yang diberikan, nilai default akan dipakai.

```
def sapa(nama="Teman"):
    print(f"Halo, {nama}!")

sapa()          # Output: Halo, Teman!
sapa("Rina")    # Output: Halo, Rina!
```

Function dengan Banyak Nilai (Multiple Return Values)

```
def operasi_matematika(a, b):
    tambah = a + b
    kurang = a - b
    return tambah, kurang

hasil_tambah, hasil_kurang = operasi_matematika(10, 5)
print("Tambah:", hasil_tambah)
print("Kurang:", hasil_kurang)
```

Function dengan *args dan **kwargs

- *args digunakan untuk menerima sejumlah argumen tanpa batas (sebagai tuple).
- **kwargs digunakan untuk menerima argumen keyword (sebagai dictionary).

```
def total(*angka):
    return sum(angka)

print(total(1, 2, 3, 4))  # Output: 10
```

```
def data_mahasiswa(**kwargs):
    for key, value in kwargs.items():
        print(f"{key} : {value}")
```

```
data_mahasiswa(nama="Budi", umur=20, jurusan="Informatika")
```

Function Rekursif (Recursive Function)

Function rekursif adalah function yang **memanggil dirinya sendiri**. Biasanya dipakai untuk menyelesaikan masalah yang bisa dibagi menjadi sub-masalah yang lebih kecil, seperti faktorial, deret Fibonacci, dsb.

Contoh: Faktorial

```
def faktorial(n):
    if n == 0:
        return 1
    else:
        return n * faktorial(n - 1)

print(faktorial(5))  # Output: 120
```

Penjelasan:

- Kalau n adalah 0, maka hasilnya 1 (base case).
- Jika tidak, function memanggil dirinya sendiri sampai n = 0.

Lambda Function

Lambda function adalah function kecil tanpa nama (anonymous function) yang biasanya dipakai untuk operasi singkat. Pakainya keyword lambda.

Contoh:

```
# Function biasa
def kali(x):
    return x * 2

# Lambda version
kali_lambda = lambda x: x * 2
print(kali_lambda(5))  # Output: 10
```

Lambda sering dipakai dengan function seperti map(), filter(), dan sorted().

Contoh dengan map():

```
angka = [1, 2, 3, 4]
hasil = list(map(lambda x: x * 2, angka))
print(hasil)  # Output: [2, 4, 6, 8]
```

Penerapan Function dalam Program Mini

Contoh: Kalkulator Sederhana

```
def tambah(a, b):
    return a + b

def kurang(a, b):
    return a - b

def kali(a, b):
    return a * b

def bagi(a, b):
    if b == 0:
        return "Tidak bisa dibagi dengan nol!"
    return a / b

# Penggunaan
print("Kalkulator")
x = int(input("Masukkan angka pertama: "))
y = int(input("Masukkan angka kedua: "))
operasi = input("Pilih operasi (+, -, *, /): ")

if operasi == "+":
    print("Hasil:", tambah(x, y))
elif operasi == "-":
    print("Hasil:", kurang(x, y))
elif operasi == "*":
```

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

```
    print("Hasil:", kali(x, y))
elif operasi == "/":
    print("Hasil:", bagi(x, y))
else:
    print("Operasi tidak dikenali.")
```

Scope dalam Function (Ruang Lingkup Variabel)

Scope menentukan **di mana sebuah variabel bisa diakses**. Ada 2 jenis utama:

1. Local Scope (dalam function)

Variabel yang dideklarasikan **di dalam function**, hanya bisa diakses di dalam function itu saja.

```
def halo():
    pesan = "Halo Dunia!"
    print(pesan)

halo()
# print(pesan)  ❌ Error, karena 'pesan' cuma hidup di dalam function
```

2. Global Scope (di luar function)

Variabel yang dideklarasikan **di luar function**, bisa diakses di manapun dalam file Python.

```
nama = "Siti"  # variabel global

def sapa():
    print(f"Halo, {nama}!")

sapa()  # Output: Halo, Siti
```

Gunakan global jika ingin mengubah variabel global dari dalam function

```
x = 10
```

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

```
def ubah():
    global x
    x = 20

ubah()
print(x)  # Output: 20
```

Biasanya disarankan **hindari penggunaan global terlalu sering**, karena bisa bikin kode jadi susah dilacak.

Membuat Modular Program (Pemisahan Function ke File Lain)

Kalau kodenya mulai banyak, lebih baik dipisah jadi beberapa file. Misalnya, kamu bikin file math_utils.py:

File: math_utils.py

```
def tambah(a, b):
    return a + b

def kali(a, b):
    return a * b
```

File utama: main.py

```
import math_utils

print(math_utils.tambah(2, 3))  # Output: 5
print(math_utils.kali(4, 5))    # Output: 20
```

Jadi kode lebih rapi dan mudah dipelihara!

Kesimpulan

Konsep	**Keterangan Singkat**
def	Digunakan untuk mendeklarasikan function
return	Mengembalikan nilai dari function

Parameter & argumen	Untuk mengirim data ke function
*args / **kwargs	Untuk menerima banyak argumen
Rekursif	Function memanggil dirinya sendiri
Lambda	Function tanpa nama (singkat)
Scope	Ruang lingkup variabel
Modularisasi	Pemisahan function ke file lain

Apa itu Built-in Function?

Built-in functions adalah fungsi yang **sudah disediakan langsung oleh Python**, jadi kamu bisa langsung pakai **tanpa harus membuat atau mengimport** apa-apa.

Contoh Built-in Functions yang Sering Dipakai

Berikut ini beberapa yang populer dan sering digunakan:

Function	**Kegunaan**
print()	Menampilkan output ke layar
input()	Menerima input dari pengguna
len()	Menghitung panjang (jumlah item) dari string, list, dll
type()	Mengetahui tipe data
int(), str()	Mengubah tipe data ke integer atau string
sum()	Menjumlahkan semua nilai dalam list atau tuple
max() / min()	Mencari nilai tertinggi atau terendah
abs()	Mengembalikan nilai absolut
round()	Membulatkan angka
range()	Membuat deret angka
sorted()	Mengurutkan data

Contoh Penggunaan:

len() – menghitung panjang data

```
nama = "Python"
print(len(nama))  # Output: 6
```

type() – cek tipe data

```
angka = 10
print(type(angka))  # Output: <class 'int'>
```

sum(), max(), min()

```
data = [1, 5, 3, 9, 2]
print(sum(data))  # Output: 20
print(max(data))  # Output: 9
print(min(data))  # Output: 1
```

input() – ambil data dari user

```
nama = input("Masukkan nama kamu: ")
print("Halo", nama)
```

int(), float(), str()

```
angka_str = "100"
angka = int(angka_str)
print(angka + 50)  # Output: 150
```

range() dan list()

```
print(list(range(1, 6)))  # Output: [1, 2, 3, 4, 5]
```

sorted() – mengurutkan data

```
angka = [4, 1, 7, 3]
print(sorted(angka))  # Output: [1, 3, 4, 7]
```

Tips

- Kamu bisa lihat semua built-in functions Python di dokumentasi resminya: https://docs.python.org/3/library/functions.html

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

Project Mini: Aplikasi Daftar Kontak

Fitur:

1. Tambah kontak (nama & nomor telepon)
2. Tampilkan semua kontak
3. Cari kontak berdasarkan nama
4. Hapus kontak
5. Keluar dari program

Contoh Implementasi:

```
# Data disimpan dalam dictionary
kontak = {}

def tambah_kontak(nama, nomor):
    kontak[nama] = nomor
    print(f"Kontak '{nama}' berhasil ditambahkan.")

def tampilkan_kontak():
    if not kontak:
        print("Belum ada kontak.")
    else:
        print("Daftar Kontak:")
        for nama, nomor in kontak.items():
            print(f"- {nama} : {nomor}")

def cari_kontak(nama):
    if nama in kontak:
        print(f"Kontak ditemukan: {nama} - {kontak[nama]}")
    else:
        print(f"Kontak '{nama}' tidak ditemukan.")

def hapus_kontak(nama):
    if nama in kontak:
```

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

```
        del kontak[nama]
        print(f"Kontak '{nama}' berhasil dihapus.")
    else:
        print(f"Kontak '{nama}' tidak ditemukan.")

# Menu utama
while True:
    print("\n===== Menu Daftar Kontak =====")
    print("1. Tambah Kontak")
    print("2. Tampilkan Semua Kontak")
    print("3. Cari Kontak")
    print("4. Hapus Kontak")
    print("5. Keluar")

    pilihan = input("Pilih menu (1-5): ")

    if pilihan == "1":
        nama = input("Masukkan nama kontak: ")
        nomor = input("Masukkan nomor telepon: ")
        tambah_kontak(nama, nomor)
    elif pilihan == "2":
        tampilkan_kontak()
    elif pilihan == "3":
        nama = input("Masukkan nama yang ingin dicari: ")
        cari_kontak(nama)
    elif pilihan == "4":
        nama = input("Masukkan nama yang ingin dihapus: ")
        hapus_kontak(nama)
    elif pilihan == "5":
        print("Terima kasih! Program selesai.")
        break
    else:
```

```
print("Pilihan tidak valid. Coba lagi.")
```

16
Object-Oriented Programming (OOP)

Apa Itu OOP (Object-Oriented Programming)?

OOP adalah **gaya pemrograman** yang berfokus pada **objek**. Objek terdiri dari **data (disebut properti/atribut)** dan **fungsi (disebut method)** yang bekerja bersama.

Konsep Dasar OOP:

1. **Class**: Cetakan / blueprint untuk membuat objek
2. **Object**: Hasil nyata dari class
3. **Constructor** (__init__) : Method khusus saat objek dibuat
4. **Attribute**: Data atau variabel dalam objek
5. **Method**: Fungsi dalam class
6. **Inheritance**: Pewarisan dari class lain
7. **Encapsulation**: Penyembunyian data
8. **Polymorphism**: Satu nama method, banyak perilaku

Class

Pengertian Class

Dalam Python, **class** adalah **template** atau **blueprint** untuk membuat **objek** (object). Objek merupakan representasi nyata dari class yang berisi **data (atribut)** dan **perilaku (metode/fungsi)**.

Class memungkinkan kita untuk membuat tipe data sendiri yang kompleks, sehingga kita bisa mengelompokkan data dan fungsi yang berkaitan ke dalam satu kesatuan.

Fungsi Class

1. **Membuat Objek**

   Dengan class, kita bisa menciptakan banyak objek yang memiliki struktur data dan perilaku yang sama.

2. **Mengelompokkan Data dan Fungsi**

Atribut dan metode bisa dikelompokkan dalam satu wadah sehingga kode lebih terorganisir dan mudah dipahami.

3. **Reusability (Penggunaan Ulang)**
   Kita bisa membuat class sekali, dan menggunakannya berkali-kali tanpa harus menulis ulang kode.

4. **Encapsulation (Penyembunyian Data)**
   Class memungkinkan pengaturan visibilitas data sehingga hanya fungsi tertentu yang bisa mengaksesnya.

5. **Inheritance (Pewarisan)**
   Class dapat diturunkan ke class lain untuk mewarisi atribut dan metode yang sama, sehingga mempermudah pengembangan program kompleks.

Pengertian Object

Objek (Object) dalam Python adalah **instansiasi** atau **salinan nyata** dari sebuah **class**. Ketika sebuah class didefinisikan, class tersebut hanya berupa blueprint atau template. Untuk menggunakan class tersebut, kita perlu membuat objek dari class tersebut.
Objek adalah entitas yang memiliki **atribut** (data) dan **metode** (fungsi) yang didefinisikan dalam class. Setiap objek bisa memiliki nilai atribut yang berbeda meskipun mereka diciptakan dari class yang sama.

Fungsi Object:

1. **Menyimpan Data**: Objek menyimpan data dalam bentuk atribut (misalnya, merk dan warna dalam contoh di atas).
2. **Melakukan Aksi**: Objek bisa melakukan aksi melalui metode yang didefinisikan dalam class-nya (seperti metode jalan() pada contoh di atas).
3. **Memudahkan Pengelolaan Kode**: Dengan objek, kita dapat membuat berbagai instance dengan data yang berbeda, yang meningkatkan modularitas dan penggunaan ulang kode.

Contoh Dasar: Class dan Object

```
class Mobil:
```

```
    def __init__(self, merk, warna):
        self.merk = merk
        self.warna = warna

    def jalan(self):
        print(f"{self.merk} warna {self.warna} sedang berjalan.")

# Membuat objek dari class Mobil
mobil_saya = Mobil("Toyota", "Merah")  # objek pertama
mobil_anda = Mobil("Honda", "Biru")   # objek kedua

# Mengakses metode dan atribut objek
mobil_saya.jalan()  # Output: Toyota warna Merah sedang berjalan.
mobil_anda.jalan()  # Output: Honda warna Biru sedang berjalan.
```

Penjelasan:

1. **Class Mobil**: Merupakan blueprint atau template untuk membuat objek "Mobil".
2. **Objek mobil_saya dan mobil_anda**: Merupakan instansi nyata dari class "Mobil". Masing-masing objek ini memiliki nilai atribut yang berbeda, yaitu merk dan warna.
3. **Metode jalan():** Metode ini dapat dipanggil untuk setiap objek untuk menampilkan informasi terkait objek tersebut.

Inheritance (Pewarisan)

Pengertian Inheritance (Pewarisan)

Inheritance atau **pewarisan** dalam Python adalah konsep di mana sebuah **class** dapat mewarisi atribut dan metode dari **class lain**. Dengan pewarisan, kita dapat menciptakan class baru yang mengambil karakteristik (atribut dan metode) dari class yang sudah ada, sambil tetap memungkinkan kita untuk menambah atau memodifikasi fitur tertentu sesuai kebutuhan.

Pada dasarnya, pewarisan memungkinkan class baru (yang disebut **subclass** atau **child class**) untuk mewarisi dan memperluas kemampuan dari class lama (yang disebut **superclass** atau **parent class**).

Fungsi Inheritance:

1. **Penggunaan Ulang Kode**: Pewarisan memungkinkan kita untuk menggunakan kembali kode yang sudah ada tanpa perlu menulis ulang.
2. **Modularitas dan Organisasi**: Dengan pewarisan, kita dapat membagi fungsionalitas ke dalam class terpisah yang lebih modular dan mudah dipelihara.
3. **Extensibility (Kemampuan untuk Diperluas)**: Dengan inheritance, kita bisa menambahkan fungsionalitas baru pada class yang sudah ada tanpa mengubah class aslinya.
4. **Polimorfisme**: Inheritance memungkinkan polimorfisme, di mana metode yang sama dapat berperilaku berbeda tergantung pada jenis objek yang memanggilnya (seperti contoh metode suara() di atas).

Pentingnya Pewarisan dalam OOP:

- **Meningkatkan Efisiensi Kode**: Tidak perlu menduplikasi kode. Class-child dapat menggunakan atau memodifikasi metode yang diwariskan dari class-parent.
- **Mengurangi Kompleksitas**: Menyusun kode dalam hierarki class yang lebih sederhana dan mudah dikelola.

Sintaks Pewarisan

Untuk membuat pewarisan, kita cukup mendeklarasikan nama class parent dalam tanda kurung saat mendefinisikan class child.

```
class Parent:
    def __init__(self, nama):
        self.nama = nama

    def tampilkan_nama(self):
        print(f"Nama: {self.nama}")

class Child(Parent):
    def __init__(self, nama, umur):
        # Memanggil constructor dari class parent
```

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

```
        super().__init__(nama)
        self.umur = umur

    def tampilkan_umur(self):
        print(f"Umur: {self.umur}")
```

Contoh Penggunaan Inheritance

```
class Hewan:
    def __init__(self, nama):
        self.nama = nama

    def suara(self):
        pass  # Method ini akan didefinisikan lebih lanjut di subclass

class Kucing(Hewan):
    def suara(self):
        print(f"{self.nama} berkata Meow!")

class Anjing(Hewan):
    def suara(self):
        print(f"{self.nama} berkata Bark!")

# Membuat objek
kucing = Kucing("Tom")
anjing = Anjing("Buddy")

# Mengakses metode yang diwariskan dan metode yang didefinisikan di subclass
kucing.suara()  # Output: Tom berkata Meow!
anjing.suara()  # Output: Buddy berkata Bark!
```

Penjelasan:

1. **Class Hewan (Parent)**: Ini adalah class dasar yang memiliki atribut nama dan metode suara() yang belum didefinisikan secara spesifik.

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

2. **Class Kucing dan Anjing (Child)**: Kedua class ini mewarisi class Hewan, tetapi mereka mendefinisikan metode suara() mereka sendiri sesuai kebutuhan.
3. **Keyword super()**: Digunakan untuk memanggil konstruktor atau metode dari class induk (parent) agar class child dapat mewarisi atribut atau metode dari class parent.

Dengan inheritance, kita dapat membangun sistem yang lebih fleksibel dan mudah dikembangkan

Encapsulation (Menyembunyikan data)

Pengertian Encapsulation (Penyembunyian Data)

Encapsulation atau **penyembunyian data** adalah konsep dalam pemrograman berorientasi objek (OOP) di mana **data (atribut)** dan **metode (fungsi)** yang beroperasi pada data tersebut dikelompokkan ke dalam satu unit yang disebut **class**, dan data internal tersebut disembunyikan dari akses langsung oleh luar class. Akses terhadap data dilakukan hanya melalui **metode** yang disediakan oleh class itu sendiri.

Dengan kata lain, encapsulation memungkinkan kita untuk menyembunyikan rincian implementasi dari class dan hanya mengekspose apa yang perlu diketahui pengguna class tersebut. Ini bertujuan untuk **mengontrol** bagaimana data diubah dan diakses.

Fungsi Encapsulation:

1. **Keamanan Data**: Dengan menyembunyikan data, kita bisa memastikan bahwa data hanya diubah atau diakses melalui cara yang aman dan terkontrol.
2. **Pengontrolan Akses**: Dengan menggunakan getter dan setter, kita bisa mengatur siapa yang bisa mengakses dan mengubah data, serta melakukan validasi saat data diubah.
3. **Meningkatkan Keterbacaan dan Pemeliharaan Kode**: Kode menjadi lebih terorganisir karena data dan fungsinya terbungkus dalam satu unit yang jelas, memudahkan debugging dan pengembangan.
4. **Mengurangi Risiko Kesalahan**: Menyembunyikan implementasi internal mencegah penggunaan yang salah oleh pihak luar class yang tidak memahami struktur data secara rinci.

5. **Modularitas**: Dengan menggunakan encapsulation, kita dapat mengubah implementasi data internal tanpa mengubah antarmuka pengguna, yang menjadikannya lebih fleksibel untuk pengembangan lebih lanjut.

Pentingnya Encapsulation dalam OOP:

- Menjaga integritas data.
- Membantu memisahkan bagian yang dapat diubah (implementasi internal) dan bagian yang tetap (antarmuka pengguna).
- Menjaga kode tetap terstruktur dengan baik dan mudah dikelola.

Cara Encapsulation Bekerja di Python

- **Atribut Privat**: Dalam Python, kita dapat menggunakan **underscore tunggal** (_) atau **double underscore** (__) di depan nama atribut atau metode untuk menunjukkan bahwa atribut atau metode tersebut bersifat privat (tidak boleh diakses langsung dari luar class).
- **Metode Getter dan Setter**: Untuk mengakses atau memodifikasi atribut privat, kita dapat menggunakan metode **getter** (untuk mengambil nilai) dan **setter** (untuk mengubah nilai).

Contoh Implementasi Encapsulation

```
class AkunBank:
    def __init__(self, pemilik, saldo):
        self.pemilik = pemilik
        self.__saldo = saldo  # Atribut __saldo bersifat privat

    # Getter untuk mengambil saldo
    def get_saldo(self):
        return self.__saldo

    # Setter untuk mengubah saldo
    def set_saldo(self, jumlah):
        if jumlah >= 0:
            self.__saldo = jumlah
```

```
        else:
            print("Saldo tidak bisa negatif!")

    # Metode untuk menambah saldo
    def tambah_saldo(self, jumlah):
        if jumlah > 0:
            self.__saldo += jumlah
        else:
            print("Jumlah harus positif!")

    # Metode untuk menarik saldo
    def tarik_saldo(self, jumlah):
        if 0 < jumlah <= self.__saldo:
            self.__saldo -= jumlah
        else:
            print("Saldo tidak cukup atau jumlah tidak valid!")
```

Contoh Penggunaan

```
# Membuat objek dari class AkunBank
akun = AkunBank("John Doe", 1000)

# Mengakses saldo melalui metode getter
print(akun.get_saldo())  # Output: 1000

# Mengubah saldo melalui metode setter
akun.set_saldo(2000)
print(akun.get_saldo())  # Output: 2000

# Menambah saldo
akun.tambah_saldo(500)
print(akun.get_saldo())  # Output: 2500
```

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

```
# Menarik saldo
akun.tarik_saldo(1000)
print(akun.get_saldo())  # Output: 1500
```

Penjelasan:

1. **Atribut Privat (__saldo)**: Atribut __saldo disembunyikan menggunakan double underscore agar tidak bisa diakses langsung dari luar class.
2. **Metode Getter (get_saldo())**: Digunakan untuk mengambil nilai saldo.
3. **Metode Setter (set_saldo())**: Digunakan untuk mengubah nilai saldo, dengan validasi agar saldo tidak negatif.
4. **Metode Tambah dan Tarik Saldo**: Menyediakan cara aman untuk menambah dan menarik saldo dengan validasi.

Polymorphism (Bentuk Beragam)

Pengertian Polymorphism

Polymorphism berasal dari bahasa Yunani yang berarti "**banyak bentuk**". Dalam konteks **OOP (Object-Oriented Programming)**, polymorphism adalah kemampuan suatu **objek** untuk **mengambil berbagai bentuk**, terutama saat **metode yang sama digunakan untuk objek dari class yang berbeda**, tetapi dengan **perilaku yang berbeda**.

Dengan kata lain, polymorphism memungkinkan kita menggunakan **nama metode yang sama** di berbagai class, namun dengan **implementasi yang berbeda**. Ini membuat kode lebih fleksibel, efisien, dan mudah dikembangkan.

Fungsi dan Manfaat Polymorphism

1. **Menyederhanakan Kode**

   Kita bisa menggunakan metode/fungsi yang sama untuk berbagai tipe objek.
2. **Mempermudah Pemeliharaan dan Perluasan Program**

   Saat menambahkan class baru, kita tidak perlu mengubah kode yang sudah ada.
3. **Mendukung Prinsip OOP**

   Terutama prinsip **Open-Closed Principle** (kode terbuka untuk ekstensi, tertutup untuk modifikasi).

4. **Meningkatkan Reusability (penggunaan ulang)**
   Kode menjadi lebih fleksibel dan bisa digunakan kembali di berbagai kondisi.
5. **Mendukung Desain yang Modular dan Dinamis**
   Karena objek bisa diganti-ganti selama mendukung metode yang sama.

Jenis Polymorphism dalam Python

1. **Polymorphism dengan Inheritance (Pewarisan)**
   Saat class child mewarisi dari class parent dan **mengubah (override)** metode dari parent.

```
class Hewan:
    def suara(self):
        print("Hewan mengeluarkan suara")

class Kucing(Hewan):
    def suara(self):
        print("Kucing berkata Meow")

class Anjing(Hewan):
    def suara(self):
        print("Anjing berkata Guk-guk")

# Menggunakan polymorphism
def bunyikan_suara(hewan):
    hewan.suara()

# Objek dari class yang berbeda, tapi metode yang sama
bunyikan_suara(Kucing())  # Output: Kucing berkata Meow
bunyikan_suara(Anjing())  # Output: Anjing berkata Guk-guk
```

2. **Polymorphism dengan Fungsi atau Metode Umum**
   Metode yang sama bisa digunakan oleh objek berbeda tanpa mewarisi dari class yang sama, asal mereka punya metode yang sama.

```
class Burung:
    def terbang(self):
        print("Burung terbang di udara")

class Pesawat:
    def terbang(self):
        print("Pesawat terbang di langit")

# Fungsi umum
def coba_terbang(objek):
    objek.terbang()

coba_terbang(Burung())    # Output: Burung terbang di udara
coba_terbang(Pesawat())   # Output: Pesawat terbang di langit
```

Konsep	Penjelasan
Polymorphism	Kemampuan objek untuk menggunakan metode yang sama dengan perilaku berbeda
Tujuan	Meningkatkan fleksibilitas, efisiensi, dan kemampuan ekspansi dari program
Contoh	Metode suara() pada class Kucing, Anjing, dan Hewan berbeda hasilnya meskipun namanya sama

Rangkuman

Konsep	Keterangan Singkat
class	Cetakan untuk membuat objek
object	Instansi nyata dari class
__init__()	Constructor untuk inisialisasi
self	Referensi ke objek itu sendiri
inheritance	Pewarisan class dari class lain
encapsulation	Membuat data privat atau tersembunyi

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

polymorphism	Satu method bisa punya banyak perilaku

Project Mini: Sistem Manajemen Mahasiswa

Fitur:

1. Tambah mahasiswa
2. Tampilkan daftar mahasiswa
3. Cari mahasiswa berdasarkan NIM
4. Hapus Mahasiswa berdasarkan NIM
5. Edit Mahasiswa (nama dan jurusan) berdasarkan NIM
   Setelah edit atau hapus, data otomatis disimpan ke file
6. Keluar dari program

Listing Programnya:

```
import json
import os

class Mahasiswa:
    def __init__(self, nama, nim, jurusan):
        self.nama = nama
        self.nim = nim
        self.jurusan = jurusan

    def tampilkan_info(self):
        print(f"Nama    : {self.nama}")
        print(f"NIM     : {self.nim}")
        print(f"Jurusan : {self.jurusan}")
        print("----------------------------")

    def to_dict(self):
        return {"nama": self.nama, "nim": self.nim, "jurusan": self.jurusan}

class SistemManajemen:
```

```
def __init__(self, filename):
    self.daftar_mahasiswa = []
    self.filename = filename
    self.load_data()

def tambah_mahasiswa(self, nama, nim, jurusan):
    mhs = Mahasiswa(nama, nim, jurusan)
    self.daftar_mahasiswa.append(mhs)
    self.simpan_data()
    print(f"Mahasiswa {nama} berhasil ditambahkan.")

def tampilkan_semua(self):
    if not self.daftar_mahasiswa:
        print("Belum ada data mahasiswa.")
    else:
        print("\n=== Daftar Mahasiswa ===")
        for mhs in self.daftar_mahasiswa:
            mhs.tampilkan_info()

def cari_mahasiswa(self, nim):
    for mhs in self.daftar_mahasiswa:
        if mhs.nim == nim:
            print("\nMahasiswa ditemukan:")
            mhs.tampilkan_info()
            return mhs
    print(f"Mahasiswa dengan NIM {nim} tidak ditemukan.")
    return None

def hapus_mahasiswa(self, nim):
    for mhs in self.daftar_mahasiswa:
        if mhs.nim == nim:
            self.daftar_mahasiswa.remove(mhs)
```

```
            self.simpan_data()
            print(f"Mahasiswa dengan NIM {nim} berhasil dihapus.")
            return
        print(f"Mahasiswa dengan NIM {nim} tidak ditemukan.")

    def edit_mahasiswa(self, nim):
        mhs = self.cari_mahasiswa(nim)
        if mhs:
            nama_baru = input("Masukkan nama baru: ")
            jurusan_baru = input("Masukkan jurusan baru: ")
            mhs.nama = nama_baru
            mhs.jurusan = jurusan_baru
            self.simpan_data()
            print("Data mahasiswa berhasil diubah.")

    def simpan_data(self):
        with open(self.filename, "w") as file:
            data = [mhs.to_dict() for mhs in self.daftar_mahasiswa]
            json.dump(data, file, indent=4)

    def load_data(self):
        if os.path.exists(self.filename):
            with open(self.filename, "r") as file:
                try:
                    data = json.load(file)
                    for item in data:
                        mhs = Mahasiswa(item["nama"], item["nim"], item["jurusan"])
                        self.daftar_mahasiswa.append(mhs)
                except json.JSONDecodeError:
                    print("File JSON rusak atau kosong. Memulai dari data kosong.")

# Program Utama
```

```
sistem = SistemManajemen("mahasiswa.json")

while True:
    print("\n=== Menu Manajemen Mahasiswa ===")
    print("1. Tambah Mahasiswa")
    print("2. Tampilkan Semua Mahasiswa")
    print("3. Cari Mahasiswa")
    print("4. Edit Mahasiswa")
    print("5. Hapus Mahasiswa")
    print("6. Keluar")

    pilihan = input("Pilih menu (1-6): ")

    if pilihan == "1":
        nama = input("Masukkan nama: ")
        nim = input("Masukkan NIM: ")
        jurusan = input("Masukkan jurusan: ")
        sistem.tambah_mahasiswa(nama, nim, jurusan)
    elif pilihan == "2":
        sistem.tampilkan_semua()
    elif pilihan == "3":
        nim = input("Masukkan NIM yang dicari: ")
        sistem.cari_mahasiswa(nim)
    elif pilihan == "4":
        nim = input("Masukkan NIM mahasiswa yang akan diedit: ")
        sistem.edit_mahasiswa(nim)
    elif pilihan == "5":
        nim = input("Masukkan NIM mahasiswa yang akan dihapus: ")
        sistem.hapus_mahasiswa(nim)
    elif pilihan == "6":
        print("Data disimpan. Sampai jumpa!")
        break
```

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

```
else:
    print("Pilihan tidak valid, coba lagi.")
```

17
Exceptions & Errors

Dalam Python, **Exceptions** dan **Errors** adalah bagian penting dari sistem penanganan kesalahan. Keduanya digunakan untuk menunjukkan bahwa terjadi sesuatu yang tidak normal selama eksekusi program. Berikut penjelasannya:

Pengertian Exceptions & Errors dalam Python

1. **Error**

   Error adalah kesalahan yang terjadi ketika interpreter Python tidak dapat memahami atau menjalankan kode. Error biasanya muncul sebelum program dijalankan sepenuhnya (saat proses *compiling* atau *runtime*), dan menyebabkan program berhenti secara tiba-tiba.

   Jenis-jenis error yang umum:

   - **SyntaxError** → Kesalahan dalam penulisan sintaksis Python
     Contoh: if x == 5 (tanpa tanda titik dua :)
   - **IndentationError** → Kesalahan dalam penggunaan indentasi
     Contoh: print("Hello") tanpa indentasi di dalam blok kode
   - **NameError** → Variabel yang belum dideklarasikan
     Contoh: print(x) saat x belum didefinisikan
   - **TypeError** → Operasi dilakukan pada tipe data yang tidak sesuai
     Contoh: 3 + "hello"

2. **Exception**

   Exception adalah kondisi abnormal yang terjadi saat program sedang berjalan (*runtime*) yang bisa ditangani oleh programmer menggunakan blok try-except. Exceptions tidak selalu fatal dan bisa diantisipasi.

   Contoh Exception:

   - **ZeroDivisionError** → Pembagian dengan nol
   - **FileNotFoundError** → File yang dicari tidak ditemukan
   - **ValueError** → Nilai tidak sesuai dengan tipe yang diharapkan

Fungsi Exceptions & Errors dalam Python

1. **Menjaga kestabilan program**

   Dengan exception handling, kita bisa mencegah program crash secara tiba-tiba.

2. **Mendeteksi dan menangani kesalahan secara spesifik**

   Programmer bisa membuat *respon khusus* untuk jenis kesalahan tertentu.

3. **Meningkatkan pengalaman pengguna**

   Pengguna tidak akan melihat pesan error yang membingungkan, tapi akan diarahkan dengan cara yang lebih baik.

Contoh Penggunaan Exception

```
try:
    angka = int(input("Masukkan angka: "))
    hasil = 10 / angka
    print("Hasil:", hasil)
except ZeroDivisionError:
    print("Tidak bisa membagi dengan nol.")
except ValueError:
    print("Input harus berupa angka.")
```

Struktur Penanganan Exception di Python

Python menggunakan blok berikut untuk menangani exception:

```
try:
    # Kode yang mungkin menyebabkan exception
except JenisException:
    # Penanganan jika exception terjadi
else:
    # Kode ini dijalankan jika TIDAK ada exception
finally:
    # Kode ini SELALU dijalankan, baik ada exception atau tidak
```

Contoh Lengkap:

```
try:
```

E-LMK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

```
    angka = int(input("Masukkan angka: "))
    hasil = 10 / angka
except ZeroDivisionError:
    print("Error: Tidak bisa membagi dengan nol.")
except ValueError:
    print("Error: Input harus berupa angka.")
else:
    print("Hasil pembagian:", hasil)
finally:
    print("Program selesai dijalankan.")
```

Membuat Exception Sendiri (Custom Exception)

Kita bisa membuat exception sendiri untuk kasus tertentu menggunakan class:

```
class NilaiNegatifError(Exception):
    pass

def cek_nilai(n):
    if n < 0:
        raise NilaiNegatifError("Nilai tidak boleh negatif!")

try:
    nilai = int(input("Masukkan nilai: "))
    cek_nilai(nilai)
    print("Nilai yang dimasukkan:", nilai)
except NilaiNegatifError as e:
    print("Kesalahan:", e)
```

Daftar Beberapa Exception Bawaan Python

Exception	**Penjelasan**
ZeroDivisionError	Pembagian dengan nol
ValueError	Nilai tidak sesuai tipe data
TypeError	Tipe data tidak sesuai saat operasi

IndexError	Akses indeks yang di luar jangkauan
KeyError	Akses kunci yang tidak ada di dictionary
FileNotFoundError	File yang dicari tidak ditemukan
ImportError	Gagal mengimpor modul
AttributeError	Atribut yang tidak ditemukan

18
Venv & PIP

Venv (Virtual Environment)

Pengertian:

venv adalah modul bawaan di Python yang digunakan untuk membuat **lingkungan virtual** (virtual environment).

Lingkungan virtual ini memungkinkan kamu untuk memiliki **instalasi Python dan pustaka (library)** yang terisolasi dari sistem utama.

Fungsi:

- Menghindari konflik versi pustaka antar proyek.
- Menjaga kebersihan dan keteraturan lingkungan pengembangan.
- Memungkinkan setiap proyek memiliki versi dependensi (library) masing-masing.

Contoh Penggunaan:

```
# Membuat lingkungan virtual
python -m venv nama_env

# Mengaktifkan lingkungan virtual
# Windows
nama_env\Scripts\activate
# Linux/MacOS
source nama_env/bin/activate

# Menonaktifkan lingkungan virtual
deactivate
```

PIP (Pip Installs Packages)

Pengertian:

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

pip adalah **package manager** resmi untuk Python. Digunakan untuk **menginstal dan mengelola** pustaka dari Python Package Index (PyPI).

Fungsi:

- Menginstal library dari PyPI.
- Menghapus dan memperbarui library.
- Menyimpan daftar dependensi proyek dengan requirements.txt.

Contoh Penggunaan:

```
# Install package
pip install nama_paket

# Uninstall package
pip uninstall nama_paket

# Melihat package yang terinstal
pip list

# Simpan daftar package ke file
pip freeze > requirements.txt

# Install dari file requirements
pip install -r requirements.txt
```

Langkah Praktis Menggunakan venv + pip

1. **Membuat Proyek Python Baru**

   Misalnya kamu ingin membuat proyek bernama toko_online.

   ```
   mkdir toko_online
   cd toko_online
   ```

2. **Membuat Virtual Environment**

   ```
   python -m venv venv
   ```

Ini akan membuat folder bernama venv di dalam proyekmu yang menyimpan Python dan pip terisolasi.

3. **Mengaktifkan Virtual Environment**
   - **Windows:**

     ```
     venv\Scripts\activate
     ```

   - Linux / MacOS:

     ```
     source venv/bin/activate
     ```

     Setelah diaktifkan, kamu akan lihat nama environment (venv) muncul di awal baris terminalmu.
4. **Install Library dengan pip**

   Misalnya kamu butuh Flask:

   ```
   pip install flask
   ```

   pip akan menginstal Flask hanya di dalam environment venv, **bukan ke sistem Python global**.
5. **Simpan Daftar Package**

   Kalau kamu ingin menyimpan semua package yang diinstal supaya bisa dipakai di tempat lain atau oleh teman satu tim:

   ```
   pip freeze > requirements.txt
   ```
6. **Install dari requirements.txt**

   Temanmu tinggal jalankan ini di komputernya:

   ```
   pip install -r requirements.txt
   ```

   Ini akan otomatis install semua package yang dibutuhkan proyekmu.
7. **Keluar dari Virtual Environment**

   Setelah selesai kerja:

   ```
   deactivate
   ```

Ringkasan Hubungan venv dan pip:

Fitur	venv	pip
Fungsi utama	Buat lingkungan virtual	Install dan kelola package
Tipe alat	Bawaan Python	Package manager
Kegunaan	Isolasi lingkungan proyek	Instalasi dan manajemen library

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

Digunakan bersama?	✅ Ya	✅ Ya

TEKNIK KOMPUTER DAN JARINGAN
SMK NEGERI SUKASARI

19
File Operations

Pengertian File Operations

File operations dalam Python adalah serangkaian proses untuk **membaca**, **menulis**, **mengedit**, dan **menghapus** data dari atau ke file di sistem penyimpanan komputer (seperti file teks .txt, file CSV, dan lain-lain). Python menyediakan built-in function untuk bekerja dengan file melalui fungsi open().

Mode File dalam Python

Saat membuka file, kita bisa menentukan mode operasinya:

Mode	Keterangan
'r'	Read (membaca file)
'w'	Write (menulis file, isi lama dihapus)
'a'	Append (menambahkan data ke akhir file)
'x'	Create file baru, error jika sudah ada
'b'	Binary mode (digabung dengan yang lain, contoh: 'rb')
't'	Text mode (default)

Fungsi-Fungsi Umum dalam File Operations

Berikut adalah beberapa fungsi dan metode yang sering digunakan dalam operasi file:

Fungsi / Metode	Deskripsi
open(filename, mode)	Membuka file
file.read()	Membaca seluruh isi file
file.readline()	Membaca satu baris file
file.readlines()	Membaca seluruh isi file per baris, sebagai list
file.write(string)	Menulis string ke file
file.writelines(list)	Menulis daftar string ke file
file.close()	Menutup file

Contoh Sederhana

```
# Membuka dan membaca file
file = open("data.txt", "r")
isi = file.read()
print(isi)
file.close()
```

```
# Menulis ke file
file = open("data.txt", "w")
file.write("Halo, ini file baru!")
file.close()
```

```
# Menambahkan isi ke file
file = open("data.txt", "a")
file.write("\nIni tambahan baris.")
file.close()
```

Dengan Context Manager (Cara yang Direkomendasikan)

```
with open("data.txt", "r") as file:
    isi = file.read()
    print(isi)
```

Menggunakan with akan secara otomatis menutup file setelah selesai digunakan.

Jenis-Jenis Operasi File Berdasarkan Kebutuhan

1. **Membaca File (Read)**

```
with open("data.txt", "r") as file:
    isi = file.read()
    print(isi)
```

- read() membaca seluruh isi file sebagai string.
- Bisa juga pakai readline() untuk membaca baris demi baris

```
with open("data.txt", "r") as file:
```

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

```
    for baris in file:
        print(baris.strip())  # strip() menghilangkan karakter newline
```

2. **Menulis ke File (Write)**

```
with open("data.txt", "w") as file:
    file.write("Data baru\n")
```

Mode 'w' akan **menghapus isi file** jika sudah ada, lalu menulis dari awal.

3. **Menambahkan ke File (Append)**

```
with open("data.txt", "a") as file:
    file.write("Baris tambahan\n")
```

Mode 'a' akan **menambahkan** isi ke akhir file, tanpa menghapus data sebelumnya.

4. **Membaca dan Menulis Sekaligus**

```
with open("data.txt", "r+") as file:
    data = file.read()
    file.write("\nData baru setelah membaca")
```

- Mode 'r+': Membaca dan menulis.
- Mode 'w+': Menulis dari awal dan bisa membaca.
- Mode 'a+': Menambahkan dan bisa membaca.

Membaca File Baris demi Baris ke dalam List

```
with open("data.txt", "r") as file:
    baris_list = file.readlines()
    for baris in baris_list:
        print(baris.strip())
```

readlines() akan membaca semua baris dan menyimpannya ke dalam list.

Contoh Aplikasi: Menyimpan Data Input ke File

```
nama = input("Masukkan nama Anda: ")
umur = input("Masukkan umur Anda: ")
```

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

```
with open("biodata.txt", "w") as file:
    file.write(f"Nama: {nama}\n")
    file.write(f"Umur: {umur}")
```

Tips dan Best Practices

- Selalu gunakan with open(...) agar file ditutup secara otomatis.
- Gunakan strip() saat membaca file untuk membersihkan newline \n.
- Jangan lupa validasi file sebelum membacanya (misalnya cek dengan os.path.exists()).
- Untuk file besar, baca baris per baris agar tidak boros memori.

Pengolahan file khusus yang sering dipakai dalam dunia nyata seperti CSV, JSON, dan Binary Files.

1. **File CSV (Comma Separated Values)**
   CSV biasanya digunakan untuk menyimpan data tabel (seperti Excel). Python punya modul bawaan bernama csv untuk membacanya.

   Membaca CSV

```
import csv

with open("data.csv", mode='r') as file:
    reader = csv.reader(file)
    for baris in reader:
        print(baris)
```

   Menulis ke CSV

```
import csv

with open("data.csv", mode='w', newline='') as file:
    writer = csv.writer(file)
    writer.writerow(["Nama", "Umur"])
```

```
    writer.writerow(["Dina", 23])
```

2. **File JSON (JavaScript Object Notation)**

   JSON cocok untuk menyimpan data structured seperti dictionary atau list dalam Python. Modul json juga sudah bawaan.

   Menulis JSON

```
import json

data = {"nama": "Ari", "umur": 30}

with open("data.json", "w") as file:
    json.dump(data, file)
```

   Membaca JSON

```
import json

with open("data.json", "r") as file:
    data = json.load(file)
    print(data["nama"])  # Output: Ari
```

3. **Binary Files (Misalnya: gambar, file PDF, dll)**

   Untuk file non-teks, kamu harus membukanya dalam mode **binary**: 'rb' (read binary) atau 'wb' (write binary).

   Membaca File Biner

```
with open("gambar.jpg", "rb") as file:
    isi = file.read()
    print(type(isi))  # <class 'bytes'>
```

   Menulis File Biner (Copy file)

```
with open("gambar.jpg", "rb") as sumber, open("copy.jpg", "wb") as tujuan:
```

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

```
    data = sumber.read()
    tujuan.write(data)
```

Cek File Ada atau Tidak

Kadang kita perlu memastikan file tersedia dulu:

```
import os

if os.path.exists("data.txt"):
    print("File ditemukan.")
else:
    print("File tidak ditemukan.")
```

Membuat aplikasi sederhana pakai file

Mari kita buat aplikasi sederhana yang menggunakan operasi file. Sebagai contoh, kita akan membuat aplikasi **"To-Do List"** di mana pengguna bisa menambah, melihat, dan menghapus tugas yang disimpan dalam file teks.

To-Do List Application

Fitur yang akan ada:

1. Menampilkan daftar tugas.
2. Menambah tugas baru ke file.
3. Menghapus tugas dari file.

1. **Buat File Program Python**

   Kita akan mulai dengan membuat file Python bernama todo_list.py.

```
import os

# Fungsi untuk menampilkan daftar tugas
def tampilkan_tugas():
    print("\nDaftar Tugas:")
    if os.path.exists("tugas.txt"):
```

```
        with open("tugas.txt", "r") as file:
            tugas = file.readlines()
            if tugas:
                for i, t in enumerate(tugas, start=1):
                    print(f"{i}. {t.strip()}")
            else:
                print("Tidak ada tugas.")
    else:
        print("Tidak ada file tugas ditemukan.")

# Fungsi untuk menambah tugas
def tambah_tugas():
    tugas_baru = input("Masukkan tugas baru: ")
    with open("tugas.txt", "a") as file:
        file.write(tugas_baru + "\n")
    print("Tugas berhasil ditambahkan!")

# Fungsi untuk menghapus tugas
def hapus_tugas():
    tampilkan_tugas()
    try:
        nomor_tugas = int(input("\nMasukkan nomor tugas yang ingin dihapus: "))
        with open("tugas.txt", "r") as file:
            tugas = file.readlines()

        if 1 <= nomor_tugas <= len(tugas):
            tugas.pop(nomor_tugas - 1)
            with open("tugas.txt", "w") as file:
                file.writelines(tugas)
            print("Tugas berhasil dihapus!")
        else:
            print("Nomor tugas tidak valid.")
```

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

```
    except ValueError:
        print("Input tidak valid, harus berupa angka.")

# Menu Utama
def menu():
    while True:
        print("\n--- Aplikasi To-Do List ---")
        print("1. Tampilkan Daftar Tugas")
        print("2. Tambah Tugas")
        print("3. Hapus Tugas")
        print("4. Keluar")

        pilihan = input("Pilih menu (1/2/3/4): ")

        if pilihan == '1':
            tampilkan_tugas()
        elif pilihan == '2':
            tambah_tugas()
        elif pilihan == '3':
            hapus_tugas()
        elif pilihan == '4':
            print("Keluar dari aplikasi...")
            break
        else:
            print("Pilihan tidak valid. Coba lagi!")

# Jalankan program
if __name__ == "__main__":
    menu()
```

Penjelasan Kode:

1. **tampilkan_tugas()**: Fungsi ini akan membuka file tugas.txt dan membaca isinya. Jika file ada dan berisi tugas, maka tugas akan ditampilkan satu per satu.
2. **tambah_tugas()**: Fungsi ini meminta input dari pengguna untuk menambah tugas baru ke file tugas.txt.
3. **hapus_tugas()**: Fungsi ini memungkinkan pengguna untuk menghapus tugas berdasarkan nomor yang dipilih. Setelah tugas dihapus, file akan diperbarui dengan daftar tugas yang baru.
4. **menu()**: Menu utama yang akan menampilkan pilihan untuk pengguna dan memanggil fungsi sesuai dengan pilihan mereka.

Cara Menggunakan Aplikasi:

1. Jalankan program dengan perintah:

```
python todo_list.py
```

2. Anda akan melihat menu dengan pilihan untuk melihat daftar tugas, menambah tugas, menghapus tugas, atau keluar dari aplikasi.

Contoh Tampilan Interaksi:

```
--- Aplikasi To-Do List ---
1. Tampilkan Daftar Tugas
2. Tambah Tugas
3. Hapus Tugas
4. Keluar
Pilih menu (1/2/3/4): 2
Masukkan tugas baru: Belajar Python
Tugas berhasil ditambahkan!

--- Aplikasi To-Do List ---
1. Tampilkan Daftar Tugas
2. Tambah Tugas
3. Hapus Tugas
4. Keluar
Pilih menu (1/2/3/4): 1
```

TEKNIK KOMPUTER DAN JARINGAN SMK NEGERI SUKASARI

Daftar Tugas:

1. Belajar Python